PlotExtractor

User guide

Make and edit plot grids with the crop rows inside every plot, cut one original-quality image per plot from Phase One drone photos (from converted images or straight from the IIQ raw files, down to the raw sensor values), and make tiles of a fixed ground size or along the crop rows.

raw plot images (original pixels)

middle-3-row tiles, plot 3003

PlotExtractor 2026-10-30 · DPIRD / APPN · 30 October 2026

Contents

1. What PlotExtractor does

2. Quick start: the recommended way

3. Installing and starting

4. Tab 1 · Plot Grid Generation

- 4.1 Tab 1a · Generate grid
- 4.2 Tab 1b · Edit grid
- 4.3 Tab 1c · Review crop rows

5. Tab 2 · IIQ File Conversion

6. Tab 3 · Plot Extraction

- 6.1 Which option to use
- 6.2 Tab 3a · From converted images (indirect)
- 6.3 Tab 3b · Direct from IIQ

7. Tab 4 · Tiles Generation

- 7.1 Tab 4a · Fixed ground size tiles
- 7.2 Tab 4b · Row-based tiles
- 7.3 The rows numbering
- 7.4 The recipe file
- 7.5 How the rows are found
- 7.6 Where the tiles go along the plot
- 7.7 What you get (tab 4a)
- 7.8 Other tools on this tab

8. Tab 4c · QA, and are the images really original?

9. Tab 5 · Acknowledgements

10. Keeping the original data: formats, georeference and data loss

- 10.1 What each step does to the data
- 10.2 How the GeoTIFF is georeferenced, and is anything lost?
- 10.3 Recommended settings for no data loss
- 10.4 The APPN PhaseOne_Pipeline on the same data

11. Tips and good practice

- 11.1 Test first
- 11.2 Use a small margin
- 11.3 Give the sown rows
- 11.4 Flight planning for whole plots in one photo
- 11.5 Check the QA pictures
- 11.6 Use one output scale across trials

12. Troubleshooting

Appendix: files

1. What PlotExtractor does

PlotExtractor takes a drone flight that has been processed in Agisoft Metashape and gives you a plot grid, one image per plot and training tiles. The tabs are numbered in the order you normally use them:

Tab	What it does
1 Plot Grid Generation	1a Generate grid: draws the plot shapefile on an RGB orthomosaic from the 4 trial corners and the layout, optionally with the crop rows inside every plot. 1b Edit grid: moves, reshapes, grows/shrinks, widens/lengthens (by metres or by % of each plot's size) or shifts plots; corrects the grid to the drilled rows. 1c Review crop rows: makes the crop-row lines (fixed spacing from the row count, or detected in the orthomosaic), shows them plot by plot and lets you move, add or delete rows.
2 IIQ File Conversion	Converts Phase One IIQ raw files to TIFF (8 or 16 bit), GeoTIFF (georeferenced) or JPEG.
3 Plot Extraction	One image per plot, two ways. 3a From converted images (indirect): cut from the photos the project was aligned on (Phase One JPEGs), from frames converted in tab 2 (8- or 16-bit TIFF) or from the orthomosaic. 3b Direct from IIQ: each plot's box developed straight from its IIQ raw file (approach of APPN, The University of Queensland, with our changes), or copied as raw sensor values with no development at all.
4 Tiles Generation	4a Fixed ground size tiles (e.g. 1 m x 1 m, the same on every plot; or native pixel tiles, never resampled) from the middle crop rows or the whole plot. 4b Row-based tiles: whole rows (all, middle N or chosen rows) and same-size tiles along each row. 4c QA: checks the plot images against their source, the tiles against the plot images, and the rows.
5 Acknowledgements	Credits APPN (The University of Queensland) and Phase One for the IIQ plot extraction, and what we changed.

The current grid. The grid saved in tab 1a or 1b becomes the current grid: tab 1b opens it for editing, tab 1c reviews its crop rows, and tabs 3a and 3b use it by default. Each can still use any other plots file (Browse).

Coordinate system. All outputs are written in **EPSG:7850 (GDA2020 / MGA zone 50)** by default: the grids of tab 1 (reprojected from the orthomosaic's or input file's CRS) and the georeferenced images of tabs 2 and 3. Each of these tabs has an Output CRS / Save in CRS field for another EPSG code (e.g. 7851 for zone 51). Inputs may be in any CRS. Loading an orthomosaic in tab 1a or 1b sets all these fields to the orthomosaic's CRS (a lon/lat orthomosaic gives its local MGA2020 zone, 7850 at York); you can still change them.

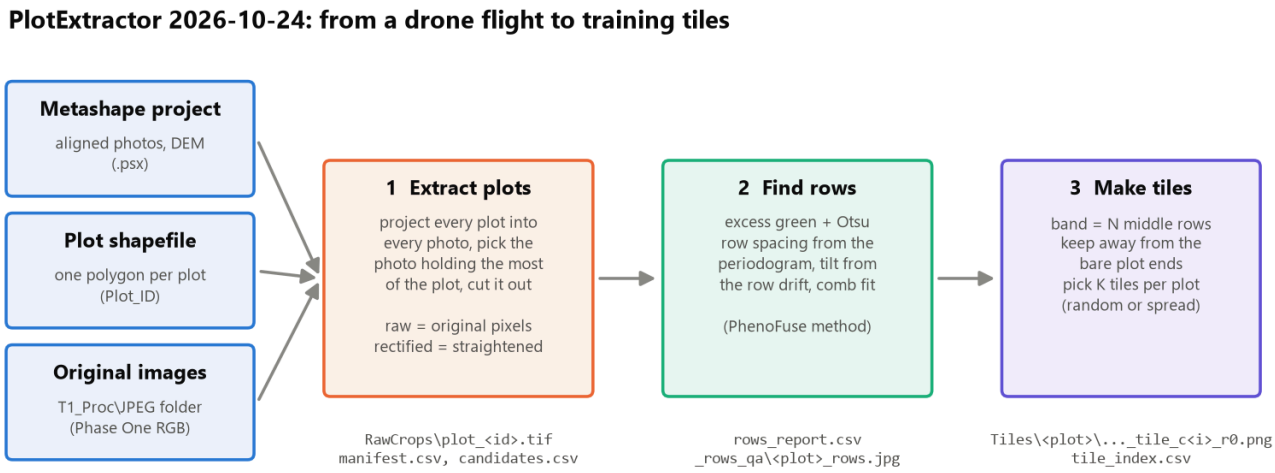

Figure 1. *The core steps (grid, plot images, tiles) and the files each step writes. The tab numbers in the figure are those of earlier versions: grid = tab 1, plot images = tab 3, tiles = tab 4.*

It was built for Phase One RGB cameras (the T1_Proc\JPEG images) but works with any camera that Metashape has aligned. With the recommended settings the plot images are **pixel-for-pixel identical to the original photos** (chapters 8 and 10, and the technical report).

2. Quick start: the recommended way

For Phase One RGB images and original-quality plot images plus middle-row training tiles:

1. Start PlotExtractor\PlotExtractor.exe.
2. **Tab 1a or 1b:** make the plot grid on the RGB orthomosaic (1a, with **Crop rows per plot** set if you want the rows too), or load your grid in 1b and fine-tune it; **Save & use** makes it the current grid.
3. **Tab 1c (optional):** press **Make rows** (fixed spacing, or detected in the orthomosaic) and page through the plots to check that the grid sits on the rows.
4. **Tab 3a (From converted images):** choose the Metashape project (.psx) and, under Raw images folder, the flight's original T1_Proc\JPEG folder; the plot grid is already filled in. Choose an output folder. (For the highest image quality, or the raw sensor data with nothing lost, use tab 3b instead, section 6.3.)
5. Set **Margin 0.10 m**, **Format GeoTIFF**, tick **Raw image crop per plot** with style **Masked to plot**, and leave **Convert raw crops to sRGB** off.
6. Set **Limit 3** and press **Run extraction** as a test. Open a few crops in RawCrops\. If they look right, set Limit back to 0 and run all plots.
7. **Tab 4 (Tiles Generation):** Plot crops folder = <output>\RawCrops; choose a tiles output folder.
8. **Tab 4a:** Tick **Tile only the middle 3 rows** and **Size the tiles to cover these rows** (buffer 0.05 m; or untick it and use Tile size 1.0 m), set **sown rows per plot** (5 at York), **Tiles per plot** (e.g. 4), pick **random** or **spread**, keep tiles **0.5 m** from the plot ends.
9. Format **tif**, Bits **same as plot image**, TIFF compression **Deflate**, tick **Georeference tiles**, **Link tiles to the source photo / IIQ**, **One folder per plot** and **QA pictures**, press **Make tiles**. Every tile holds the 3 middle rows and has the same pixel size on every plot.
10. **Tab 4b (rows):** Rows all, tick whole rows and row tiles (1 m), press **Extract rows**: <plot>\rows\ and <plot>\row_tiles\row<k>\.
11. Check _rows_qa\ (one picture per plot) and rows_report.csv, and run **tab 4c (QA)** on the output.

Recommended settings for Phase One RGB images

Extract plots	
Raw images folder	**the original T1_Proc\JPEG folder of the flight**
Format	**GeoTIFF (geotagged, lossless)**
Raw crop style	**Masked to plot (100% original pixels)**
Convert to sRGB	**off (keep the captured colours)**
Margin around plot	**0.10 m (outer rows whole when the grid is a little off)**
Limit	**3-5 for a test run, then 0 = all plots**
Training tiles	
Plot crops folder	**<output>\RawCrops**
Tile only the middle N rows	**on, N = 3**
Sown rows per plot	**the trial's sown rows (5 at York); 0 = detect**
Square tiles spanning the band	**on**
Tiles per plot	**e.g. 4 (0 = all that fit) · pick: random or spread**
Keep tiles … m from plot ends	**0.5 m**
Format / folders	**PNG · one folder per plot · QA pictures on**

Figure 2. *Recommended settings at a glance.*

Video: docs\PlotExtractor_Demo.mp4 walks through these steps in the real application (about 7 minutes, narrated; recorded with an earlier tab layout).

3. Installing and starting

- **Application:** download the release zip from github.com/appndpird/plot-extractor (Releases), unzip it and run PlotExtractor\PlotExtractor.exe. Keep the whole folder together. A console window opens next to the main window and shows progress and errors.
- **From source:** PlotExtractor.bat or python plot_extractor.py with the Python 3.11 that has the Metashape 2.3.1 module, rasterio, Pillow and pyshp.
- **Licence:** tabs 2 (GeoTIFF only), 3a and 3b need a Metashape licence (the floating licence server; VPN if off-site). Tabs 1, 4 and 5 do not.
- **Phase One SDK:** tabs 2 and 3b develop IIQ files with the Phase One ImageSDK (3b with Pixels = raw sensor values does not need it). The DPIRD build carries it; otherwise point **Phase One SDK folder** at an SDK obtained from Phase One. The status line under it must say OK.
- **Settings** are remembered in config.json next to the application.

4. Tab 1 · Plot Grid Generation

Tab 1 makes the plot grid and checks it against the crop. It has three sub-tabs: 1a Generate grid, 1b Edit grid and 1c Review crop rows. None of them needs Metashape; they work on an RGB orthomosaic GeoTIFF.

Why "grid"? A field trial is laid out as plots in ranges and rows, and a **plot grid** (or field grid) is the usual name for the shapefile with one polygon per plot (used e.g. by FIELDimageR, QGIS plot-layout tools and PhenoFuse). The tool keeps the word. After editing, the polygons need not be a perfect lattice any more - each plot can be moved and resized on its own - so in the text "grid" just means the trial's plot boundaries file. The plot rows of the layout (plots side by side in a range) are called **plots per range** in tab 1a, to keep "rows" for the **crop rows** sown inside each plot.

4.1 Tab 1a · Generate grid

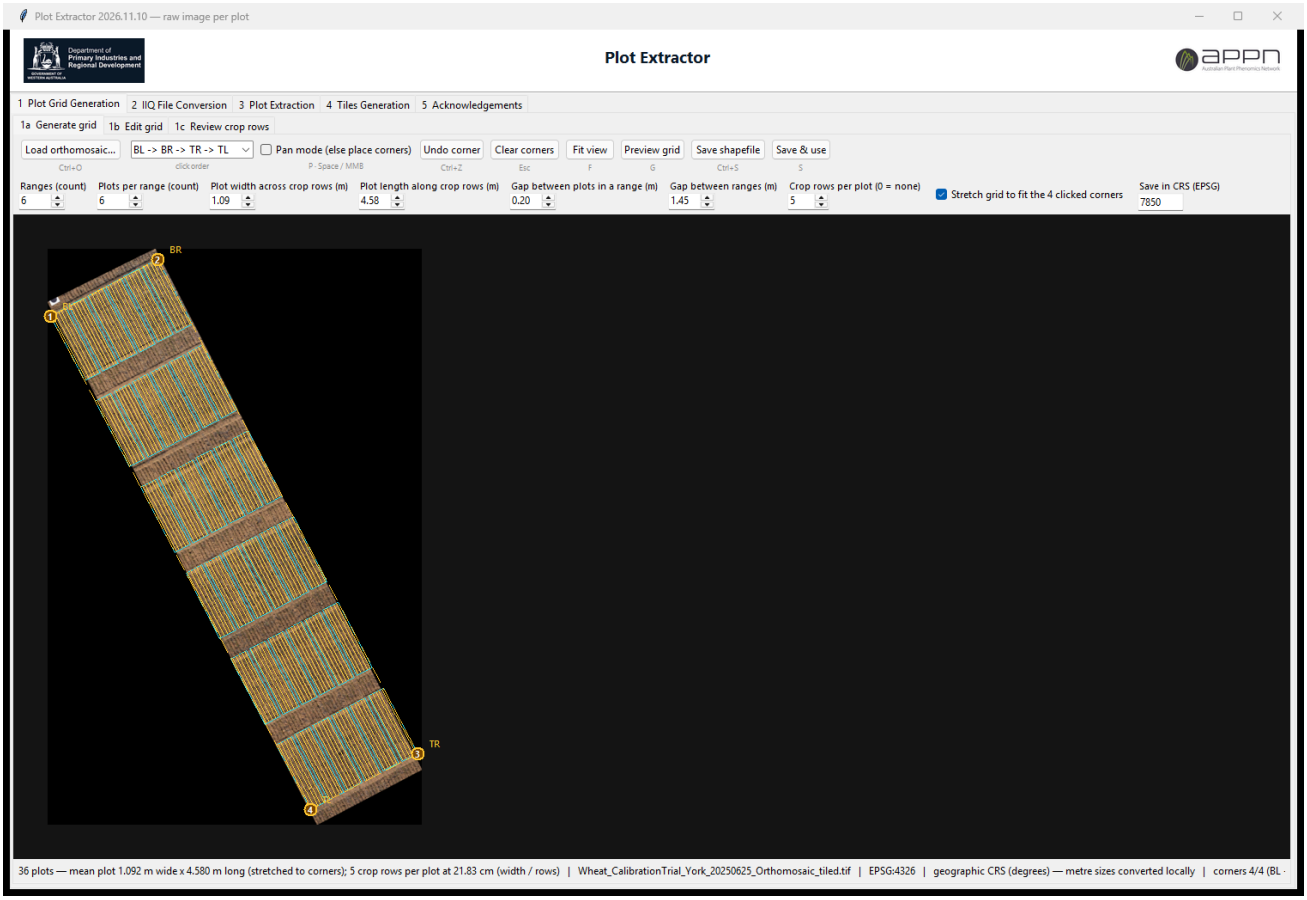

Figure 3. *Tab 1a: the 4 trial corners clicked on the York RGB orthomosaic, a 6 x 6 grid previewed with 5 crop rows per plot (yellow).*

Use it when you do not have a plot shapefile.

1. **Load orthomosaic…** (Ctrl+O). Mouse wheel zooms, Space or middle-drag pans, F fits the view.
2. Click the **4 corners of the trial** in the order shown in the drop-down. Right-click undoes the last corner.
3. Enter **Ranges**, **Plots per range**, plot width (across the crop rows) and length (along them) and the gaps. Keep **Stretch grid to fit the 4 clicked corners** on for rotated trials.
4. **Crop rows per plot** (optional): the number of rows the seeder sows in a plot (5 at York). The rows are drawn in the preview and saved as <grid>_rows.shp next to the grid.
5. **Preview grid** (G) and check the reported plot size and row spacing, then **Save & use** (S): the shapefile is written in the **Save in CRS** (default EPSG:7850), becomes the current grid and opens in tab 1b.

Crop rows from the row count. A plot seeder spaces its rows evenly, so the plot width and the row count fix every row (as in PhenoFuse 4.0.7): spacing = plot width / rows (1.05 m / 5 = 20.9 cm at York), and row k lies (k − 0.5) spacings from the plot's side edge, over the full plot length. The rows file has one line per row (Plot_ID, row_idx, v_off_m, length_m, angle_deg, source = count) and a CSV with the rows per plot. Review them in tab 1c.

4.2 Tab 1b · Edit grid

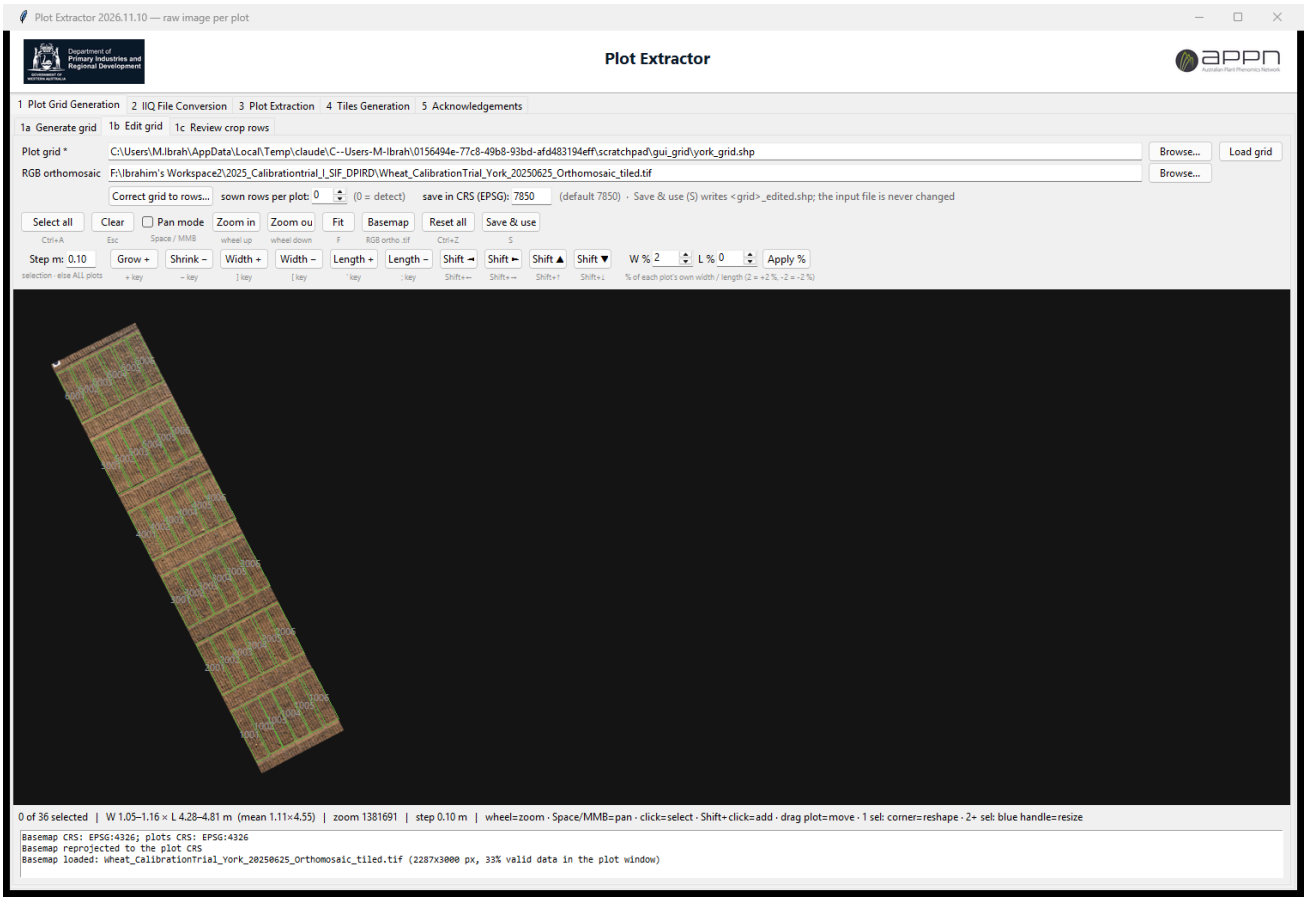

Figure 4. *Tab 1b: the York grid on the RGB orthomosaic, three plots selected; W % 2 widens each of them by 2 % of its own width.*

Edits a plot grid on the RGB orthomosaic. The file you load is never changed: Save & use writes <grid>_edited.shp next to it and makes that the current grid for tabs 1c, 3a and 3b.

1. **Plot grid:** filled with the current grid (from tab 1a, or the last grid you saved); Browse for another and press **Load grid**.
2. **RGB orthomosaic:** the trial's orthomosaic GeoTIFF; it is drawn under the plots (any CRS, reprojected to the plots). **Basemap** on the toolbar loads another one.
3. **Select:** click a plot, Shift/Ctrl-click to add, drag a box around plots, Ctrl+A for all. With nothing selected the tools act on **all** plots.
4. **Edit:** drag plots to move them; with one plot selected drag its corners; with 2+ selected drag a blue handle to resize them together. The buttons change the selection by the **Step** (m): Grow/Shrink, Width ±, Length ±, Shift ◄ ► ▲ ▼ (keys + −] [' ; and Shift+arrows).
5. **Resize by %:** type **W %** and / or **L %** (2 = 2 % wider / longer, −2 = 2 % narrower / shorter) and press **Apply %**. Each plot changes by a share of its own width / length, about its centre, as in PhenoFuse. York test: W 10 %, L −5 % gave widths × 1.10 and lengths × 0.95 on all 36 plots.

6. **Save & use** (S). The edited grid is written in the **save in CRS** (default EPSG:7850). Reset all (Ctrl+Z) goes back to the loaded grid.

Crop rows follow the plots. If the loaded grid has a rows file (<grid>_rows.shp), saving writes <grid>_edited_rows.shp too: rows from a row count are placed again in the edited plots (spacing = new width / rows); detected or hand-edited rows stay where they are on the ground.

Correct grid to rows… measures every plot against the orthomosaic and centres it on its drilled rows (set the sown rows per plot, or 0 to detect them). It writes <grid>_refit.shp with an audit CSV and asks whether to use it. It refuses when the rows run continuously across plots, because then the imagery has no plot edge to fit.

4.3 Tab 1c · Review crop rows

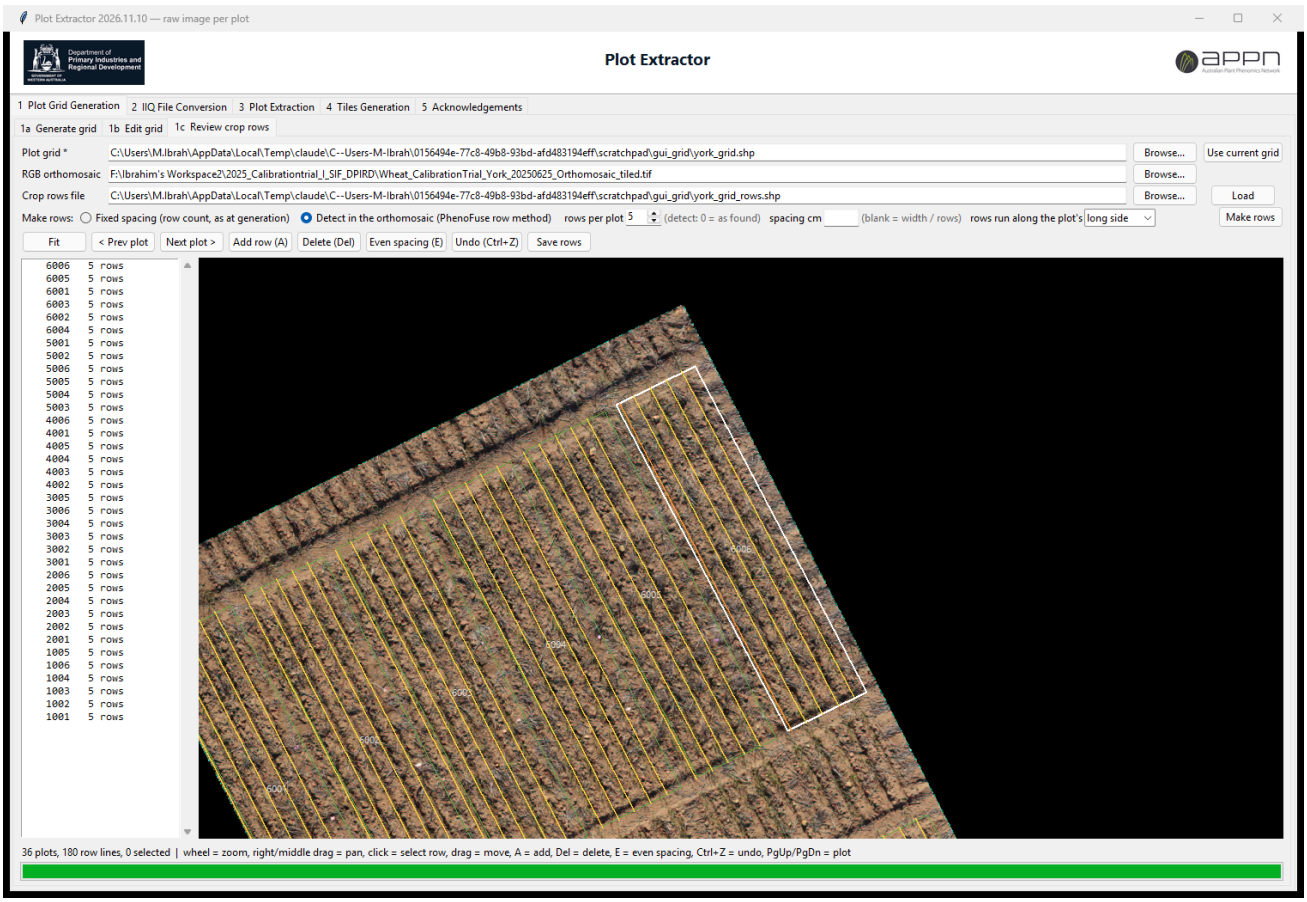

Figure 5. *Tab 1c: York plot 6006 (white) with the 5 rows detected in the orthomosaic (yellow) on the seedling rows; the plots list gives the rows of every plot.*

Makes the crop-row lines of every plot, shows them on the orthomosaic and lets you correct them, after the PhenoFuse crop-row editor.

1. **Plot grid** and **RGB orthomosaic**: **Use current grid** fills the grid from tab 1a / 1b; **Crop rows file** defaults to <grid>_rows.shp (loaded if it exists).
2. **Make rows:** choose the method and press **Make rows**:
3. **Fixed spacing (row count, as at generation):** the rows from the count, spacing = plot width / rows, or the **spacing** you type (cm; the rows are then centred in the plot). No image needed.
4. **Detect in the orthomosaic (PhenoFuse row method):** each plot is resampled into its own frame at 5 mm per pixel and the rows are found with the method of section 7.5 (spacing, tilt, comb). With **rows per plot** set, the best block of that many rows is kept; 0 keeps the rows found.

5. **Review:** the list shows every plot with its row count; plots whose count differs from the usual one are marked. Click a plot, or use **Prev / Next plot** (PgUp / PgDn); zoomed in, the orthomosaic is read at screen resolution so single rows are sharp.
6. **Edit:** click a row to select it (Shift adds), drag it across the plot, or nudge it with the arrow keys (1 cm, Shift 5 cm); **Add row** (A, then click in a plot); **Delete** (Del); **Even spacing** (E) re-spaces a plot's rows between its outer rows; **Undo** (Ctrl+Z).
7. **Save rows:** writes the rows file and a CSV (rows per plot, spacing, positions). The first save over an existing file keeps it as <rows>_before_edit.shp.

Colour	Meaning
Cyan	Row from the row count (fixed spacing)
Yellow	Row detected in the orthomosaic
Orange	Row moved or added by hand
Magenta	Selected row
Green / white outline	Plot / the plot being reviewed

York 2025 (36 plots, 5 sown rows): fixed spacing gave 180 rows at 20.9 cm in under a second. Detection on the tiled orthomosaic took 6 s for the whole trial: told 5 rows per plot it found 5 rows in all 36 plots (spacing 20.7-21.7 cm, tilt within ±0.7° of the plot axis); with rows per plot 0 it found 5 rows in 32 plots, 4 in 3 (an outer row weak) and 6 in 1 (a neighbour's row). The detected outer rows sat about 8 cm from where the fixed spacing put them - the York grid is ~9 cm off the sowing across the rows, which tab 1c makes visible plot by plot.

A striped orthomosaic (one pixel row per block) is slow to read plot by plot; **Make tiled copy** on tab 3a writes a fast, lossless copy.

5. Tab 2 · IIQ File Conversion

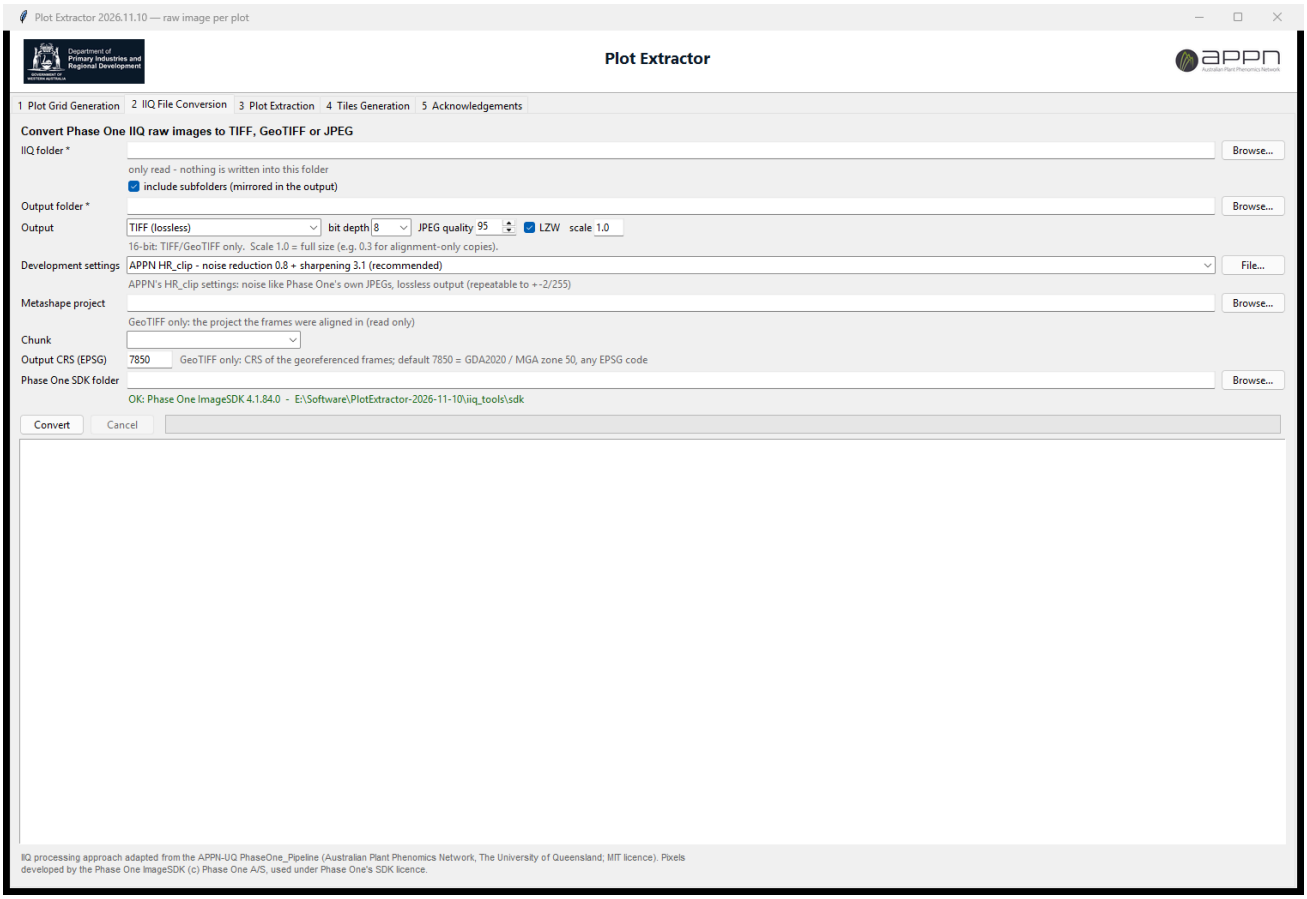

Figure 6. *Tab 2, IIQ File Conversion.*

Converts Phase One IIQ raw files to images with the Phase One ImageSDK. The IIQ folder is only read; an output folder inside it is refused.

Field	What it is	Recommended
IIQ folder	The flight's T0_Raw folder (subfolders optional, mirrored in the output)	T0_Raw
Output	TIFF (8 or 16 bit, LZW), GeoTIFF (georeferenced) or JPEG; scale 1.0 = full size	GeoTIFF, 8 bit, 1.0
Development settings	APPN HR_clip (noise reduction 0.8, sharpening 3.1), Neutral (no processing), APPN template, or a Phase One settings file	APPN HR_clip
Metashape project / Chunk	GeoTIFF only: the project the photos were aligned in. Each frame gets ground control points from its aligned camera and the DEM and an affine georeference (a lon/lat project is written in the local UTM/MGA zone)	The flight's .psx
Output CRS (EPSG)	GeoTIFF only: CRS of the georeferenced frames	7850 (default)
Phase One SDK folder	Blank = the SDK shipped with the app	Blank

Each output keeps the camera EXIF/GPS. conversion_log.csv lists every file, and IIQ_settings_used.json the settings. Checked on York 2025: a converted frame lies within 1 cm of the orthomosaic. Converted frames can

be used for plot extraction in tab 3a (indirect): a 16-bit TIFF keeps its 16 bits there, and frames of another size than the aligned photos (e.g. full size when the project was aligned on 0.3x copies) are scaled exactly.

6. Tab 3 · Plot Extraction

Tab 3 cuts one image per plot. Both sub-tabs use the same Metashape project (for the geometry), the same plot grid and the same frame choice, and write the same files (RawCrops\, manifest.csv, candidates.csv, recipe); they differ in where the pixels come from.

6.1 Which option to use

	3a From converted images (indirect)	3b Direct from IIQ
Pixels from	Images that already exist: the JPEGs the project was aligned on, TIFF / GeoTIFF frames converted in tab 2, or the orthomosaic	The IIQ raw file of the chosen photo, plot box only: developed by the Phase One SDK, or the raw sensor values
Bit depth	8 bit (JPEG); 8 or 16 bit from 16-bit TIFFs of tab 2	8 or 16 bit developed; raw sensor values 16 bit
Needs	Metashape licence	Metashape licence, IIQ files, Phase One SDK (not for raw sensor values)
Speed (York, 36 plots)	~15 s from JPEGs	16-39 s
Best for	Fast runs; when the whole flight is converted anyway; orthomosaic crops	Highest quality, and the original data with nothing lost

Same pixels either way. Converting two York IIQ frames to 16-bit TIFF in tab 2 (Neutral settings) and extracting from them in 3a gave crops **identical, pixel for pixel,** to the 3b crops of the same plots (6/6 plots, same Neutral development). From 0.5-scale TIFFs, 3a scaled the cameras by 0.5 and the crops lined up with the 3b crops within one pixel of the half-size image. 3b saves converting whole frames: it develops only each plot's box.

6.2 Tab 3a · From converted images (indirect)

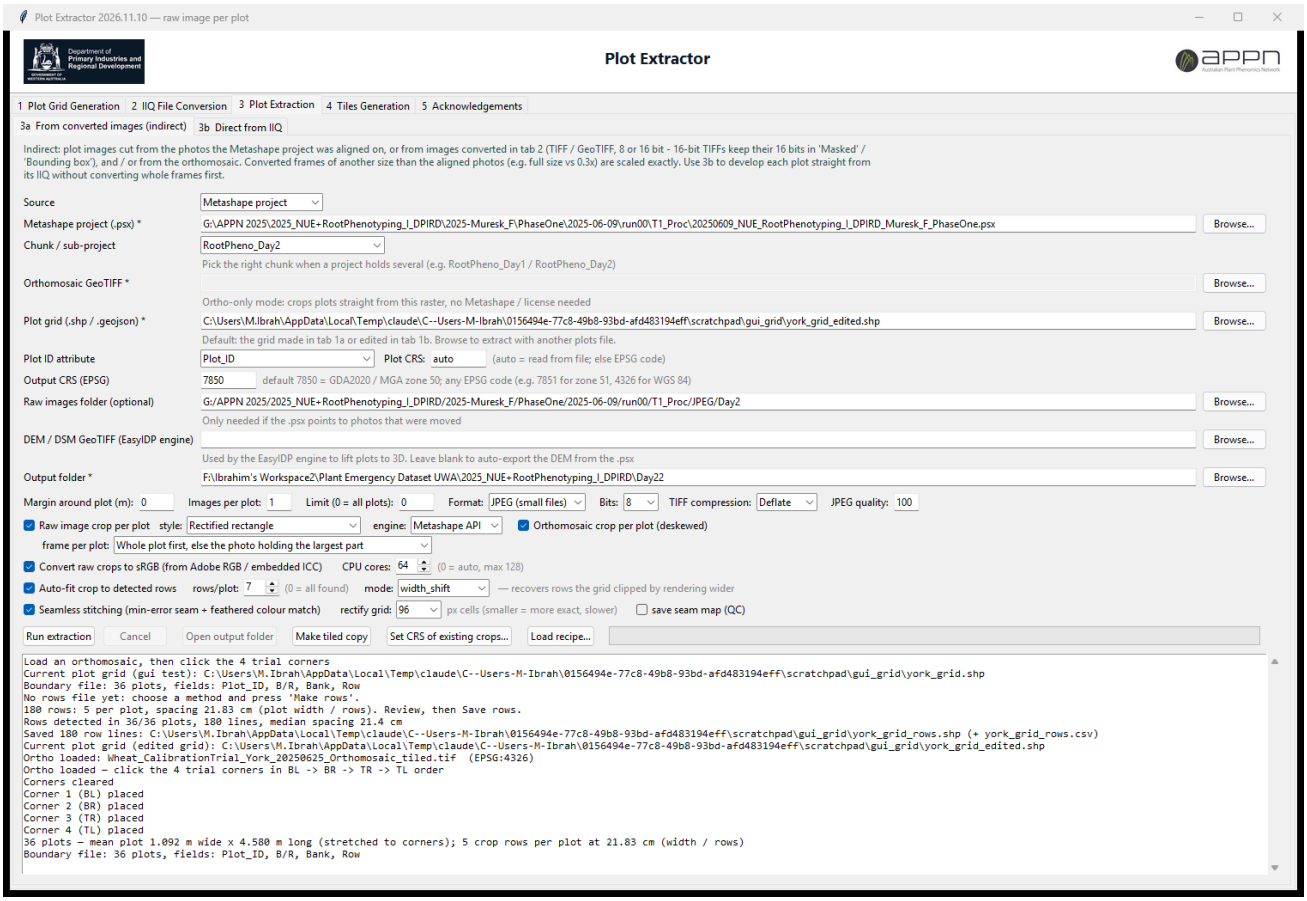

Figure 7. *Tab 3a, From converted images (indirect).*

6.2.1 The fields

Field	What it is	Recommended
Source	Metashape project (raw + ortho crops) or Orthomosaic GeoTIFF (ortho crops only, no licence)	Metashape project
Metashape project (.psx)	The flight, aligned and georeferenced, with a DEM	The run's .psx in T1_Proc
Chunk / sub-project	When a project holds several chunks	The chunk of this flight
Plot grid	Shapefile or GeoJSON, one polygon per plot, any CRS. Filled with the current grid from tab 1a/1b; Browse for another file	The current grid
Plot ID attribute	Field used to name the files (plot_<id>.tif)	Plot_ID (picked automatically)
Output CRS (EPSG)	CRS of the GeoTIFF georeference of every crop	7850 (default)
Raw images folder	Where the photos are: the JPEGs the project was aligned on, or the TIFF / GeoTIFF frames converted in tab 2 (matched by file name; another size is scaled exactly). Preferred over the paths stored in the project	T1_Proc\JPEG of the flight, or the tab 2 output

Field	What it is	Recommended
Output folder	Gets RawCrops\, manifest.csv, candidates.csv	A new folder per flight
Margin around plot (m)	Extra ground on every side of the plot	0.10 (0.2 if the grid is far off)
Images per plot	Extra views per plot (plot_<id>_2 ...)	1
Limit	Only the first N plots, for a test run	3-5 to test, then 0 = all
Format	GeoTIFF (lossless, geotagged), PNG, TIFF, JPEG (lossy)	GeoTIFF
Bits	8, or 16 when the raw images are 16-bit TIFFs from tab 2 (Masked / Bounding box styles; 8-bit output from 16-bit TIFFs = value / 257)	8, or 16 from 16-bit TIFFs
TIFF compression	Deflate, LZW or None - all lossless, identical pixels; Deflate gives the smallest files (section 6.3.1)	Deflate
Raw image crop per plot · style	How each plot is cut (section 6.2.2)	On · Masked to plot
Orthomosaic crop per plot	A straightened orthomosaic crop as well	Optional
Convert raw crops to sRGB	Converts the Adobe RGB colours of Phase One JPEGs to sRGB	Off for original pixels
CPU cores	Parallel image work; each core holds a full frame (~370 MB)	0 (auto) or 8
Frame per plot	Whole plot first, else the photo holding the largest part (or the older nadir-tier order)	Whole plot first
Auto-fit crop to detected rows	Snaps the crop to the detected rows. Needs an upright crop: with Native, Masked or Bounding box it switches the style to Rectified (resampled, may stitch photos). With rows/plot 0 it keeps every row it finds, including the outer rows of the neighbouring plots	Off. If used: rows/plot = the sown rows (e.g. 5)
Seamless stitching / rectify grid	Only for rectified crops	Defaults

Plot only, as drawn in the shapefile: Masked to plot (or Native), margin 0, Auto-fit off, sRGB off. The crop then follows the plot grid exactly, so the grid decides what is in the image: two grids of the same trial (e.g. a refit grid) give crops shifted by the same distance.

6.2.2 Raw crop styles: which one to use

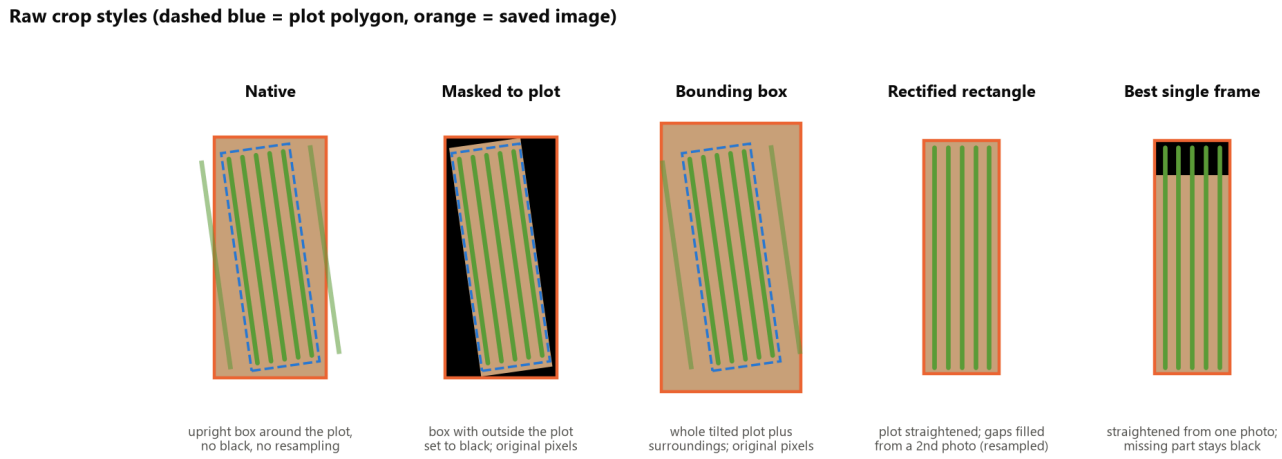

Figure 8. *The raw crop styles on a plot that is slightly tilted in the photo.*

Style	Pixels	Use it when
Masked to plot	Original, unchanged. Outside plot + margin set to black	**Recommended.** You need 100% original quality (training data, counting).
Native (no resample, no black)	Original, unchanged. Upright box, no black	You prefer no black corners and accept some neighbouring ground.
Bounding box	Original, unchanged. Whole tilted plot + surroundings	You want extra context around the plot.
Auto-straighten tilted	Original if the plot is upright, resampled if tilted > 3°	A mixed set where most plots are upright.
Rectified rectangle	Resampled (~5% lower resolution). Gaps filled from a 2nd photo	Plots must be upright and whole, e.g. plot-level measurements.
Best single frame	Resampled; what one photo misses stays black	No stitching allowed at all.

Why not always rectified? Straightening a plot means redrawing every pixel. The result looks the same but is not the original image (about 1.05 original pixels per output pixel). Masked to plot copies the pixels from the photo unchanged.

6.2.3 What you get

- RawCrops\plot_<id>.tif: one image per plot (GeoTIFF with ground control points, the photo's EXIF and colour profile).
- manifest.csv: one row per plot: source photo, crop size, raw_covered_pct (how much of the plot the image holds), frames_used, raw_style and the pixel box in the photo (src_x0 … src_y1).
- candidates.csv: every photo that sees each plot, with its coverage, so you can see the alternatives.

Figure 9. *A raw (masked) and a rectified crop of the same plot, and the raw crop against the original photo: identical pixels.*

6.2.4 Check the coverage

Sort manifest.csv by raw_covered_pct. Below 100 means no single photo holds the whole plot, so the raw image is cut at the photo edge. At York this happened to 4 of 36 plots (0.4-3.9% missing). For those plots use the rectified image if you need the whole plot, or see section 11.4 for the flight-planning fix.

6.3 Tab 3b · Direct from IIQ

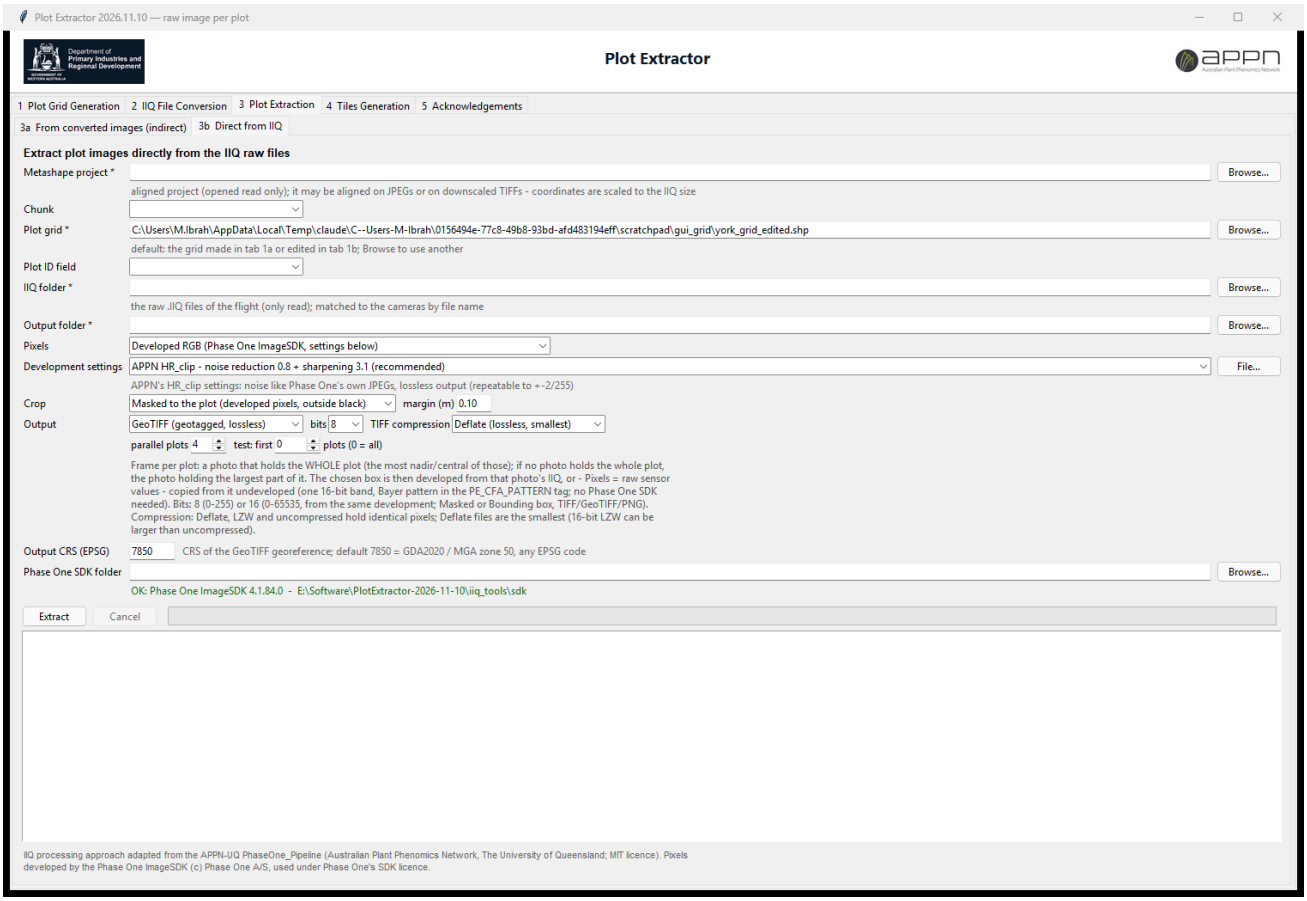

Figure 10. *Tab 3b, Direct from IIQ: Pixels, development settings, crop style, output, bits and compression.*

The same extraction as tab 3a, but each plot's box is taken straight from its IIQ raw file instead of cut from an existing image. Developed, it gives the highest image quality (less noise, more fine detail, no JPEG compression); as raw sensor values it keeps the recorded data unchanged.

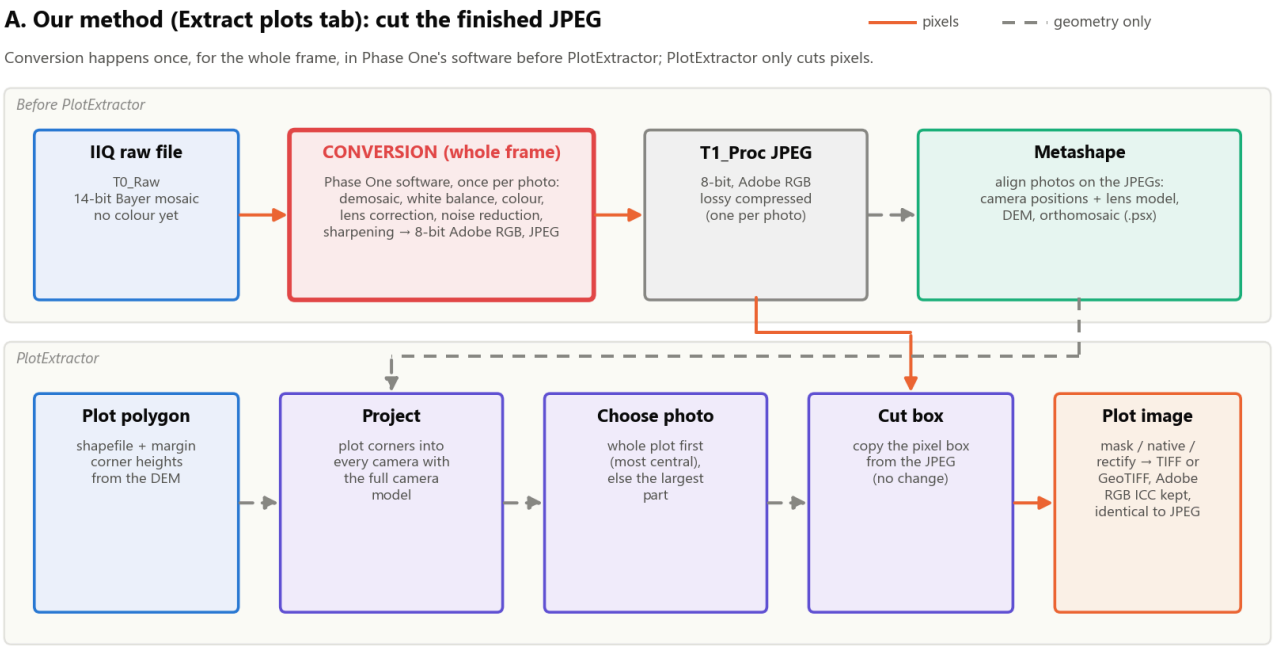

Figure 11. *Tab 3a: the conversion is done once, before the plot extraction (by Phase One's software, or in tab 2); the plot is cut from that image unchanged.*

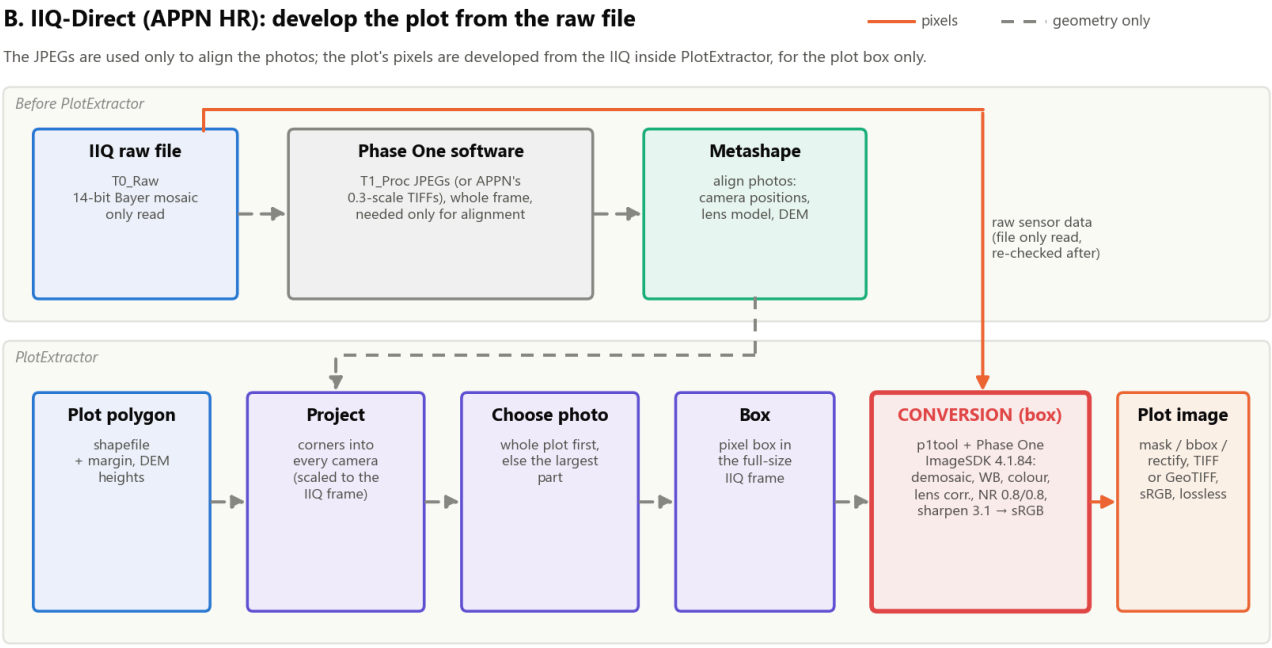

Figure 12. *Tab 3b: the JPEGs only give the geometry; the plot box is developed from the IIQ by the Phase One ImageSDK.*

1. **Metashape project:** the aligned flight (aligned on the JPEGs or on downscaled TIFFs; coordinates are scaled to the IIQ size).
2. **Plot grid:** filled with the current grid; Browse for another file. Plot ID field: Plot_ID.
3. **IIQ folder:** the flight's T0_Raw folder (only read; matched to the cameras by file name). Output folder: a new folder.
4. **Pixels:** Developed RGB (Phase One ImageSDK), or **Raw sensor values** (section 6.3.2).
5. **Development settings:** APPN HR_clip for the best-looking images; Neutral when the least-processed developed pixels matter (e.g. spectral measurements; noisier). Crop: Masked to the plot, margin 0.10 m.
6. **Output:** GeoTIFF, **bits 8** (16 for the full tonal range), **TIFF compression Deflate** (section 6.3.1).
7. Test with **first 3 plots**, check the crops, then 0 = all and **Extract**.

	Tab 3a (JPEG)	Tab 3b (IIQ, APPN HR)
Pixels	Identical to the T1_Proc JPEG	Developed from the raw sensor data
Quality (York 2025, 29 plots)	noise 3.55, detail/noise 5.4	noise 3.13, detail/noise 7.4
Colour	Phase One's JPEG colours (Adobe RGB)	SDK colours (sRGB), a little less green
Speed / needs	Fast; Metashape licence	Slower; also the IIQ files and the Phase One SDK
Repeatable	Bit for bit	Within 1-2/255 (noise reduction); Neutral bit for bit

Use one source per trial and season. Vegetation indices from JPEG-based (3a) and IIQ-developed (3b) images are not interchangeable (York: green cover 3.0% vs 2.3%, same plot ranking).

6.3.1 Bits and compression

Option	What it does	Recommended
bits 8	0-255 per channel, as the JPEGs and most training tools expect	Yes
bits 16	0-65535 per channel from the same development (divided by 257 it matches the 8-bit image to 1/255). Masked or Bounding box style; GeoTIFF, TIFF or PNG (not JPEG, not Rectified). Twice the file size; PIL reads 16-bit RGB TIFFs wrongly - use GDAL/rasterio or tifffile	Only for radiometric or exposure work
TIFF compression Deflate	Lossless (zlib + horizontal predictor). Smallest: York plot 6006, 16-bit 98 MB, 8-bit 42 MB	Yes (default)
TIFF compression LZW	Lossless. On 16-bit camera images LZW is LARGER than no compression (135 vs 126 MB); 8-bit 53 MB	Only for old readers
TIFF compression None	Uncompressed; the same pixel values (16-bit 126 MB, 8-bit 63 MB)	Only if a tool cannot read Deflate

All TIFF and GeoTIFF files are written with GDAL; the developed values are never changed after the SDK (masking only sets the pixels outside the plot to 0). The 8-bit file equals the 16-bit file divided by 257 to within one level. QA (tab 4c) re-develops every crop at its own bit depth. Compression never changes a pixel: GeoTIFF, TIFF (Deflate, LZW, none) and PNG of the same crop were identical on all 36 York plots (chapter 10).

The frame for each plot is a photo holding the whole plot (most central of those) or, if none does, the one holding the largest part. The manifest records pixel_source = iiq; IIQ_settings_used.json and IIQ_run.json record the settings and the SDK; every IIQ used is re-checked unchanged at the end.

6.3.2 Raw sensor values (no development, nothing lost)

With **Pixels = Raw sensor values** the plot box is not developed at all: the 16-bit value of every photosite under the camera's Bayer colour filter (a mosaic of red, green and blue sites) is copied from the IIQ, read with LibRaw. No black-level subtraction, demosaicing, white balance, colour matrix, tone curve, noise reduction or sharpening is applied, so this is the only output that is exactly the data the camera recorded. Each image has one band (uint16); the colour of each pixel is given by the Bayer pattern of the box, written in the PE_CFA_PATTERN tag (e.g. GBRG = first row green, blue; second row red, green). The GeoTIFF georeference is the same as for developed crops. The Phase One SDK is not needed.

- **York 2025, 36 plots:** 39 s; QA read every box again from its IIQ: 36/36 IDENTICAL. The boxes line up with the developed crops to 0.01 px. Values 0-65532, 6,000-9,000 distinct levels per plot. 1.0 GB for 36 plots (Deflate), a third of a 16-bit RGB crop set.
- **Use it** to archive the plots with nothing lost, or to develop them yourself later (e.g. in rawpy / LibRaw, or for radiometric work in linear sensor units). It is **not** a picture: viewers show a grey checkerboard, and the tiles of tab 4 need developed RGB (tab 4 refuses one-band images).
- **Styles:** Masked to the plot or Bounding box (a rectified crop would resample the values). Formats: GeoTIFF, TIFF or PNG (16-bit; not JPEG).

6.3.3 How a plot is made from the IIQ, step by step

A JPEG in T1_Proc\JPEG has already been developed once by Phase One's software, for the whole photo: 8 bit, Adobe RGB, lossy compression. The IIQ in T0_Raw holds the raw sensor data from before that. Tab 3b develops each plot from the IIQ itself; the JPEGs are only used by Metashape to align the photos (geometry, no pixels). The example is York calibration trial plot 6006 (25 June 2025).

Step 1 · Find the plot in every photo

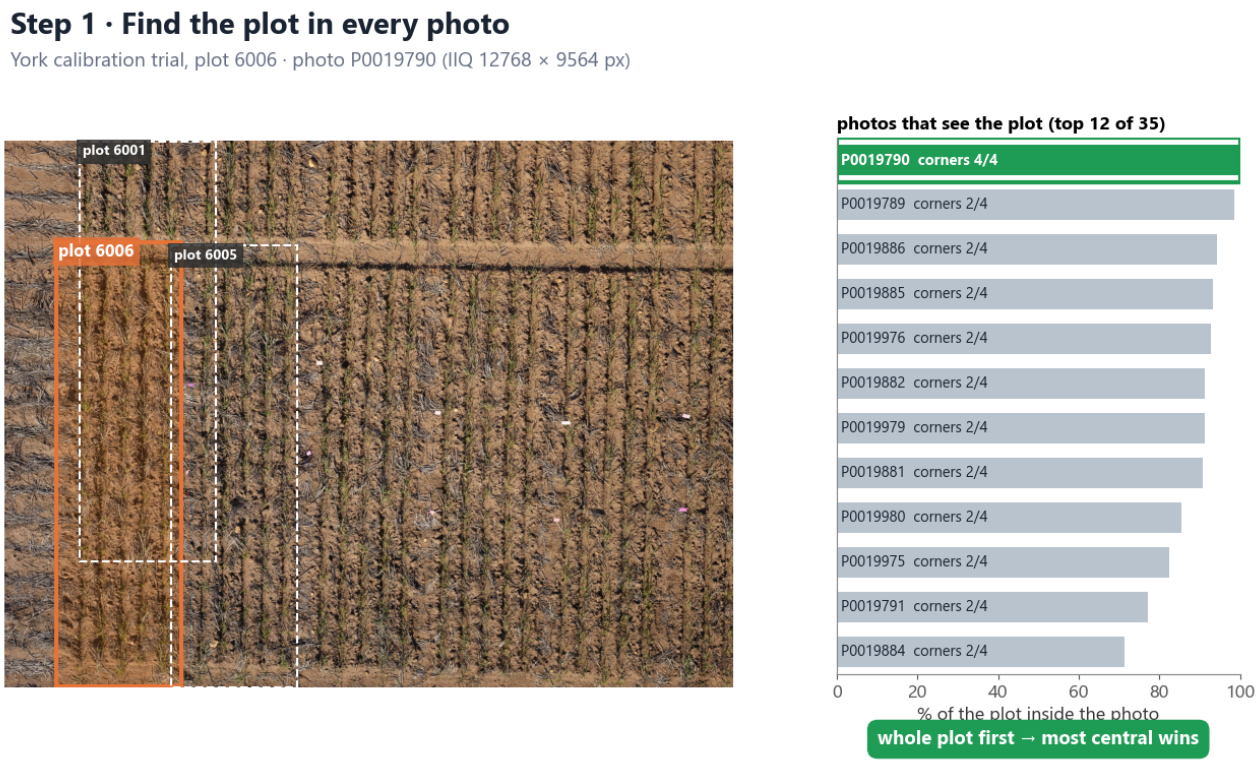

Figure 13. *Plot 6006 in photo P0019790 (left) and the photos that see it (right). Only P0019790 holds all four corners, so it is used.*

The plot polygon plus the margin is placed on the DEM and projected into every photo with Metashape's camera model, lens distortion included (coordinates are scaled to the IIQ size when the project was aligned on smaller images). For plot 6006, 35 photos see the plot. **Photos holding the whole plot come first, and among those the most central one wins** (it looks most straight down). Only when no photo holds the whole plot is the photo holding the largest part used. The candidates are listed in candidates.csv.

Step 2 · Develop only the plot's box

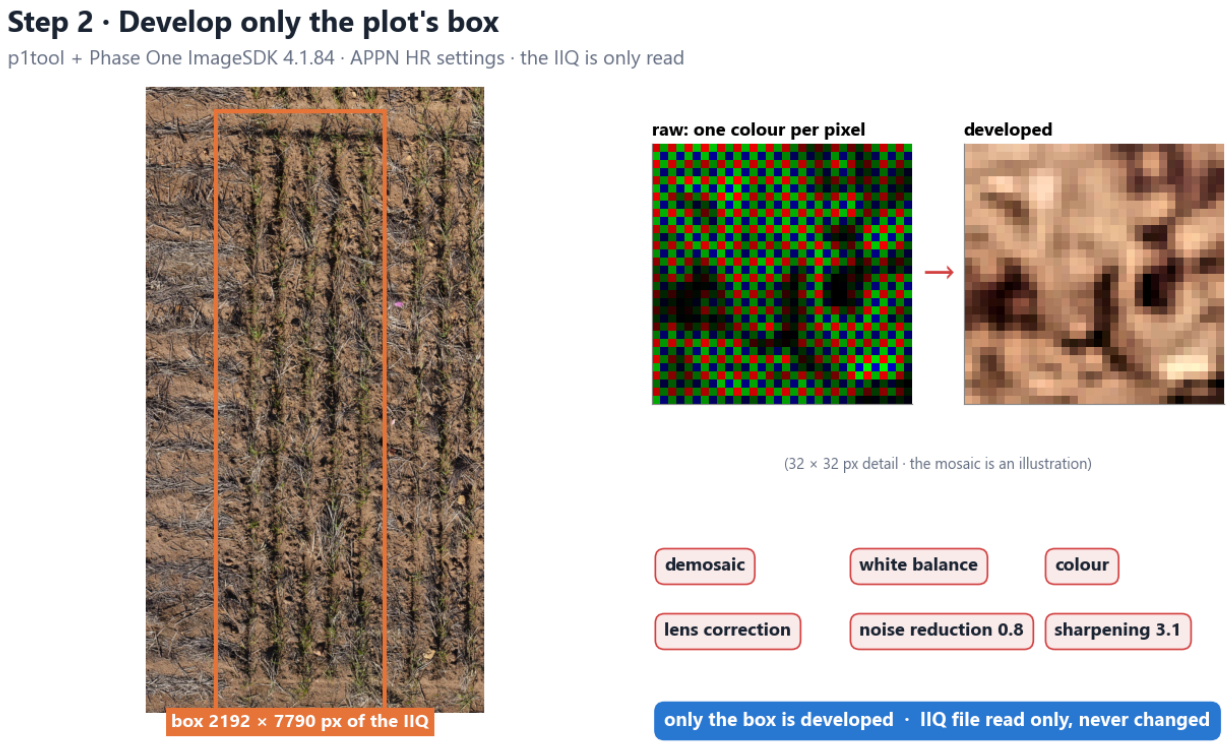

Figure 14. *The plot's box in the chosen IIQ (left) is developed by the Phase One ImageSDK. Right: a 32 x 32 px detail as the sensor saw it (one colour per pixel, an illustration) and developed.*

The plot's pixel box in that photo is passed to p1tool.exe, which uses the Phase One ImageSDK to develop only that box: rebuild full colour from the sensor's colour mosaic (demosaic: each sensor pixel saw only red, green or blue), then white balance, colour, lens correction, noise reduction (0.8) and sharpening (3.1) with the APPN HR settings, at 8 or 16 bit. Only the box is developed, so it is quick; the IIQ file is only read, never changed.

Step 3 · Mask, georeference, save

Figure 15. *The masked plot (left) and the same GeoTIFF in map coordinates, EPSG:7850 (middle).*

Pixels outside the plot polygon are set to black; no other pixel is changed. The image is written with GDAL as a GeoTIFF in GDA2020 / MGA zone 50 (EPSG:7850), with an affine georeference fitted through the plot's ground control points, so it opens in the right place in QGIS. Its tags name the IIQ (SOURCE_IMAGE) and manifest.csv records the photo and the pixel box. 8 or 16 bit and Deflate, LZW or no compression: section 6.3.1. How the georeference is made, and why it does not change a pixel: section 10.2.

7. Tab 4 · Tiles Generation

Tab 4 has three sub-tabs. **4a Fixed ground size tiles** cuts square tiles of a fixed ground size (1 m × 1 m by default) from every plot image. **4b Row-based tiles** saves whole crop rows (all of them, the middle N, or the rows you list) and cuts tiles of a fixed ground size along each row. In both, every tile has the same ground size **and** the same pixel size on every plot and every row, and each run records its settings in PlotExtractor_recipe.json (section 7.4). **4c QA** checks it all (chapter 8).

7.1 Tab 4a · Fixed ground size tiles

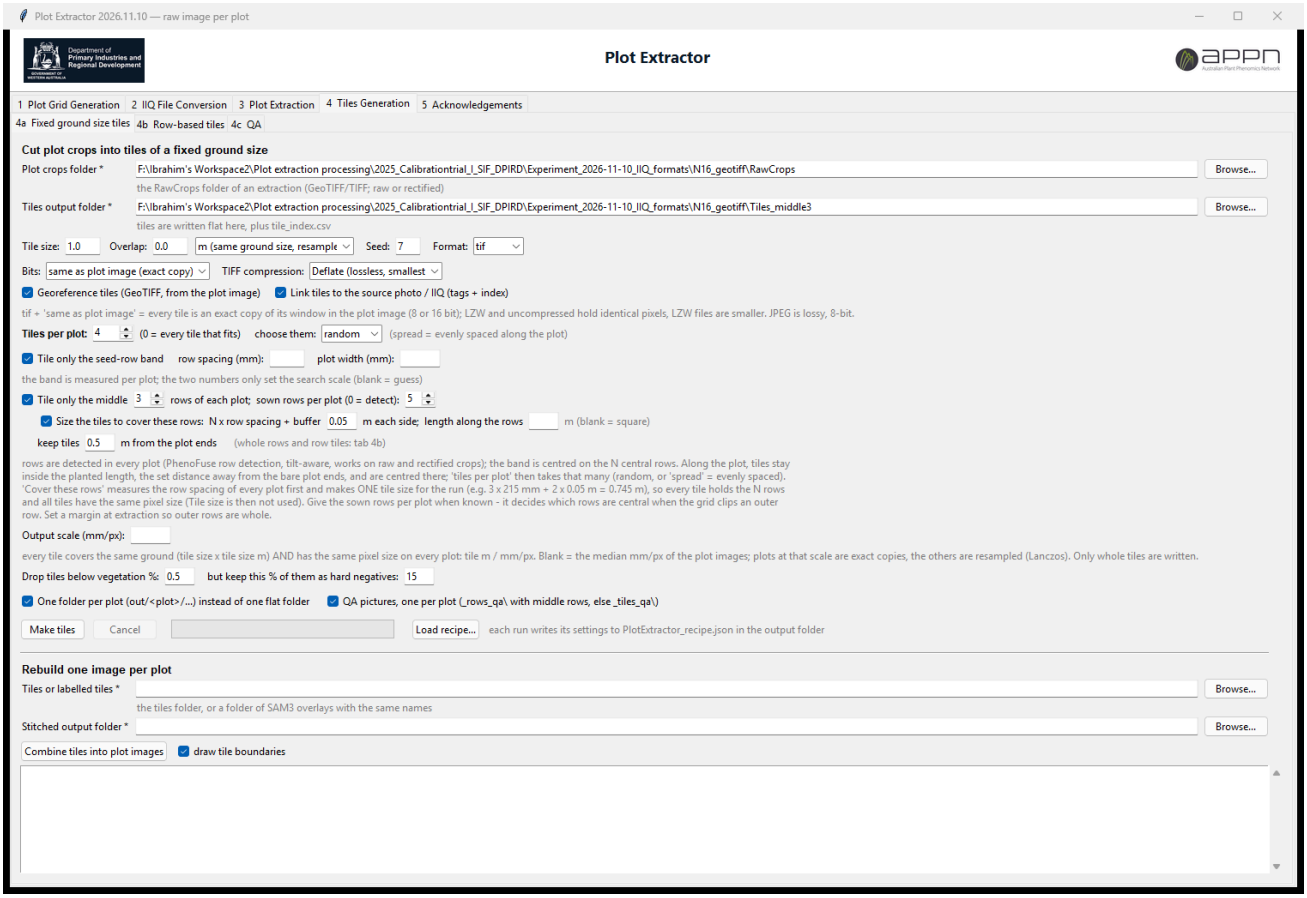

Figure 16. *Tab 4a set up for square tiles that cover the middle 3 rows (row spacing × 3 + 0.05 m each side), 4 per plot, 0.5 m away from the plot ends.*

1. **Plot crops folder:** the RawCrops folder of an extraction, raw or rectified (GeoTIFF: the ground scale comes from its georeference).
2. **Tiles output folder:** a new folder; tiles plus tile_index.csv and the recipe are written there.
3. **Tile size:** 1.0 with unit **m** for 1 m × 1 m tiles (**Overlap** 0 for tiles laid end to end); or unit **px (native pixels, never resampled)**, e.g. 1024, for tiles that are exact copies of the plot image (box below).
4. Tick **Tile only the middle N rows**, set N (3) and **sown rows per plot** (5 at York), or untick it for a grid over the seed-row band or the whole crop.
5. With middle rows, tick **Size the tiles to cover these rows** and set the **buffer** (0.05 m each side). Leave **length along the rows** blank for square tiles, or set it (e.g. 1.0 m). Tile size (m) is then not used.
6. Set **Tiles per plot** (0 = every tile that fits), **choose them** random or spread, and **keep tiles … m from the plot ends** (0.5).
7. Leave **Output scale (mm/px)** blank (the plots' median) or set it to make tiles of several runs the same pixel size.

8. Format **tif**, Bits **same as plot image**, **Deflate**, tick **Georeference tiles**, **Link tiles to the source photo / IIQ**, **One folder per plot**, **QA pictures**, then **Make tiles**.

How the size is kept the same. A plot image's ground scale (mm per pixel) comes from its georeference and differs slightly from plot to plot (each raw crop is from a different photo). The tool reads every plot's scale first and takes one **output scale** for the run: the scale of the plot nearest the median, or the value you set. A tile is then tile size / output scale pixels square on every plot (York raw: 1 m / 0.568 mm = 1761 px). On each plot the window is cut at that plot's own scale and, when it differs, resampled (Lanczos) to the tile size; plots at the output scale are exact copies. Only whole tiles are written: a plot shorter or narrower than one tile gets none (the log says so). The log ends with a size check, e.g. pixel sizes [1761] (all the same).

Option	What it does	Recommended
Tile size + unit	m: side of every tile on the ground (resampled to one pixel size). px: side in pixels of the plot image (native, never resampled; tiles of different plots cover slightly different ground)	1.0 m; 1024 px for lossless tiles
Overlap	Overlap between neighbouring tiles (same unit)	0
Output scale (mm/px)	Pixel size of the tiles; blank = the scale of the plot nearest the median. Set it to match tiles from different runs or trials	Blank, or a fixed value across trials
Tile only the middle N rows	Tiles centred on the band of the N central crop rows of each plot	On, N = 3
Size the tiles to cover these rows	Across the rows the tile is N × the median row spacing of all plots + the buffer on each side: one size for the run, holding the N rows on every plot	On
buffer (m each side)	Extra ground on each side of the N rows; raise it if the log or rows_report.csv says some plots' rows are wider than the tile	0.05
length along the rows (m)	Tile length along the rows; blank = square. Non-square tiles of plots whose rows run up-down are turned 90°, so every tile has the same width × height	Blank
sown rows per plot	How many rows were sown. Decides which rows are central when the grid clips an outer row	The trial's value (5 at York)
keep tiles … m from the plot ends	Tiles stay this far inside the plot ends (after trimming to the planted length); end to end and centred in what is left	0.5 m
Tiles per plot / choose them	0 = every tile that fits; otherwise random (Seed makes it repeatable) or spread (evenly along the plot)	As needed
Tile only the seed-row band	Grid mode (middle rows off): only the band holding the rows	On
Format, Bits, TIFF compression	tif/png lossless, jpeg lossy 8-bit; same as plot image = exact values; Deflate, LZW and None hold identical pixels	tif, same, Deflate
Georeference tiles	TIFF tiles are GeoTIFFs: the plot image's CRS and transform moved (and scaled) to the tile	On

Option	What it does	Recommended
Link tiles to the source photo / IIQ	Tags and tile_index.csv columns: source photo or IIQ and the tile's pixel box there	On
Drop tiles below vegetation %	Grid mode filter, keeping some empty tiles as hard negatives	Default
One folder per plot	out\<plot>\… instead of one flat folder	On
QA pictures	One picture per plot in every mode: _rows_qa\ (middle rows) or _tiles_qa\ (grid). Plots that got no tile have one too, with the reason written on it	On

Tiles that cover the N rows. With **Size the tiles to cover these rows** the tool first finds the rows of every plot, takes the median row spacing (York: 215 mm) and makes one tile size for the run: 3 × 215 mm + 2 × 0.05 m = 0.744 m. Every tile then holds the 3 middle rows, and all tiles have the same pixel size (0.744 m / 0.568 mm = 1310 px at York raw; 1255 px on the rectified crops). On York the 3 rows measured 618–656 mm across, so every plot kept at least 45 mm of soil on each side. rows_report.csv gives each plot's band width (band_mm), the tile width (tile_across_mm) and rows_covered; the log lists plots whose rows are wider than the tile.

Exact copies. tile_index.csv has a resampled column: False = the tile holds exactly the values of its window in the plot image (QA, tab 4c, checks this pixel by pixel). Tiles of plots at another scale are resampled to the common size, which is what makes all tiles the same size; their colour is unchanged.

Metre tiles are mostly resampled. Every raw crop comes from a different photo, so its scale differs a little (York: 0.50-0.60 mm/px). Same ground size and same pixel size on every plot is only possible by resampling: on York, 1 m tiles of the 36 Neutral 16-bit crops were exact copies on 10 of 372 tiles; 362 were resampled (Lanczos, × 0.89-1.05). When the tiles must hold the original pixel values, use unit **px**: 1024 px tiles gave 664 tiles, all exact copies, 16 bits kept. Metre tiles also need GeoTIFF plot images (or the plot width): plain TIFF, PNG and JPEG crops carry no ground scale.

7.2 Tab 4b · Row-based tiles

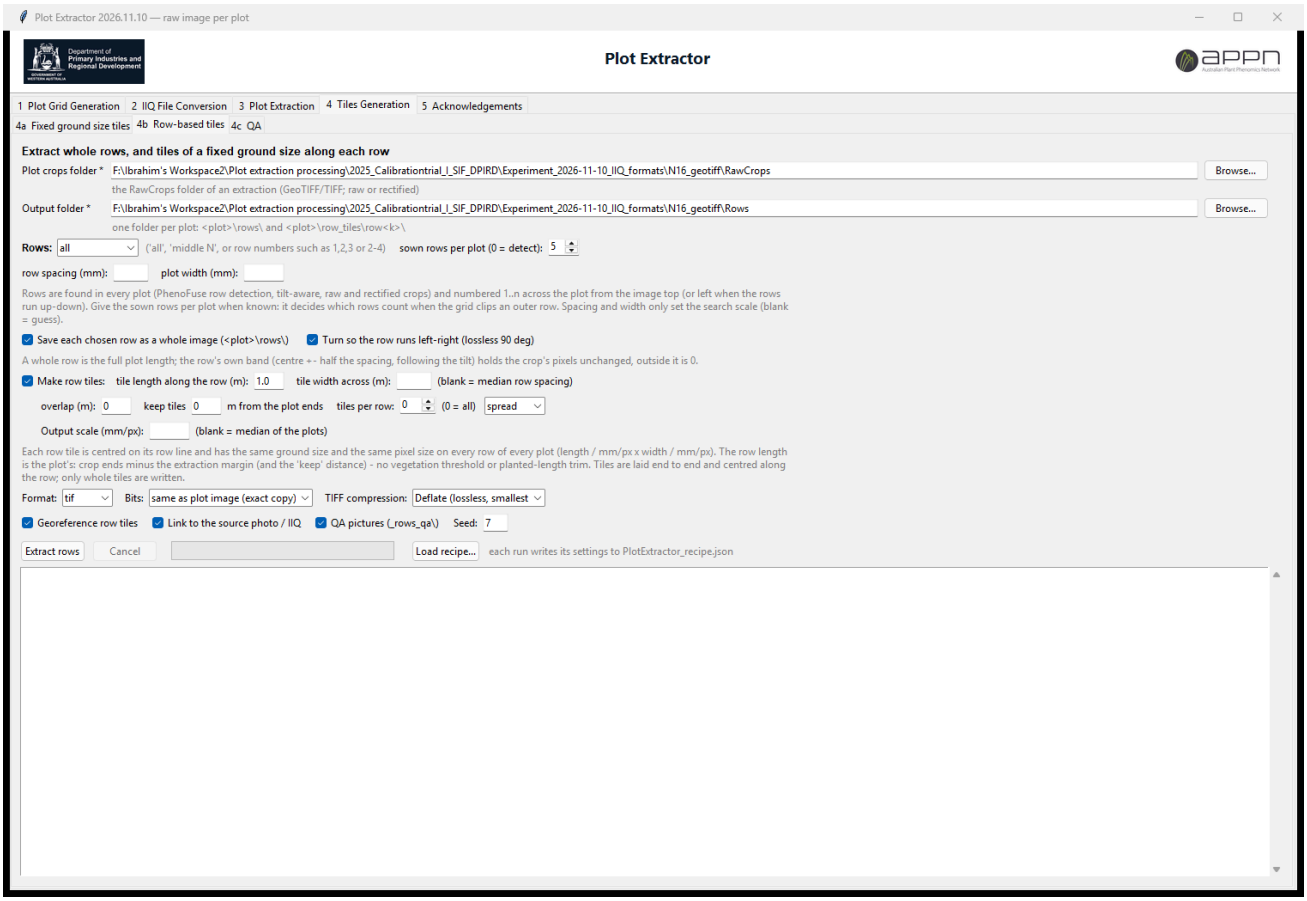

Figure 17. *Tab 4b: all rows of every plot as whole images, plus 1 m row tiles.*

1. **Plot crops folder:** <output>\RawCrops; **Output folder:** a new folder (e.g. <output>\Rows).
2. **Rows:** all, middle 3, or row numbers such as 1,2,3 or 2-4. Rows are numbered 1..n across the plot from the image top (or from the left when the rows run up-down). Set **sown rows per plot** when known.
3. Tick **Save each chosen row as a whole image** and **Turn so the row runs left-right**.
4. Tick **Make row tiles**: **tile length along the row** 1.0 m; **tile width across** blank (= the median row spacing measured over all plots, 0.215 m at York) or a value; overlap 0; keep tiles 0 m from the plot ends; tiles per row 0 (all).
5. Format, bits, compression, georeference and link as in 4a; then **Extract rows**.

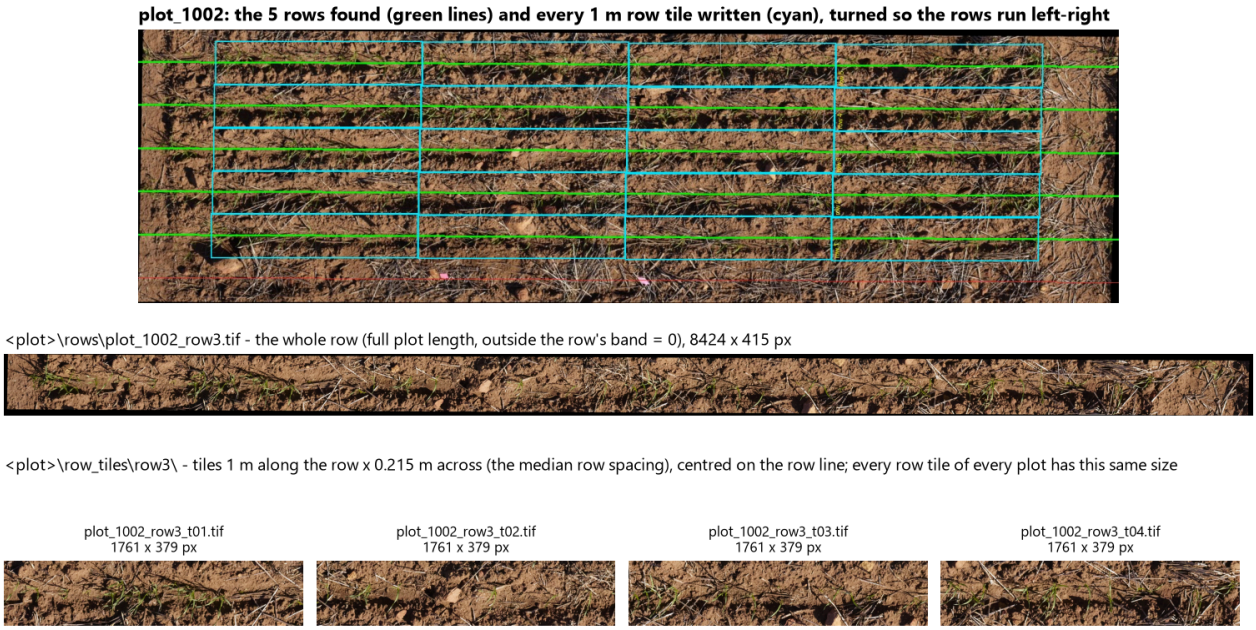

Figure 18. *York plot 1002 (raw crop, rows run up-down in the photo, turned): the 5 rows and every 1 m row tile, one whole row, and the 4 tiles of row 3. All row tiles of all plots are 1761 × 379 px.*

The row's length. A row tile run covers the row inside the plot: the crop ends minus the extraction margin (from manifest.csv) and minus the optional 'keep' distance. There is **no vegetation threshold** and no trimming to the planted length, so gaps and bare stretches of a row are kept. The tiles are laid end to end (with the overlap) and centred in that length; only whole tiles are written (York: 4.2-4.4 m of row gives 4 tiles of 1 m).

Same size everywhere. As in 4a, one output scale is used for the run, so every row tile is length / scale × width / scale pixels on every row of every plot. Each tile is centred on its row's line at the tile's middle (the line follows the tilt). With **Turn** on, plots whose rows run up-down are turned 90° (lossless, georeference turned with them), so all tiles are landscape with the row running left-right.

Folder structure:

Path	Contents
<out>\<plot>\rows\<plot>_row<k>.tif	The whole row k: full plot length, the crop's pixels inside the row's band (centre ± half a spacing, following the tilt), 0 outside
<out>\<plot>\row_tiles\row<k>\<plot>_row<k>_t<j>.tif	Row tile j of row k (1 = first along the row)
<out>\rows_index.csv	Per whole row: file, plot, row number, of rows, middle, contrast, centre and width (px), tilt, box in the crop, turned
<out>\row_tiles_index.csv	Per row tile: plot, row, tile number, box in the crop (x0, y0, w, h), output size, turned, resampled, plot and output mm/px, tile length and width (m), vegetation %, source photo link
<out>\rows_report.csv	Per plot: rows found, rows used, spacing (mm), tilt, mm/px, row length (m), rows saved, row tiles, or why it was skipped
<out>_rows_qa\<plot>_rows.jpg	The plot with its rows (green) and every row tile (cyan)
<out>\PlotExtractor_recipe.json	The settings of this run (and of the extraction the crops came from)

Option	What it does	Recommended
Rows	all, middle N, or a list (1,2,3 / 2-4). A listed row the plot does not have is skipped and reported	all
sown rows per plot	Rows sown per plot (0 = the run of rows found)	The trial's value
Save each chosen row as a whole image	<plot>\rows\ images	On
Turn so the row runs left-right	90° lossless turn for plots whose rows run up-down	On
tile length along the row (m)	Length of every row tile	1.0
tile width across (m)	Width of every row tile; blank = the median row spacing of the plots	Blank
overlap (m), keep tiles … m from the plot ends	Overlap between row tiles; distance kept from the plot ends	0, 0
tiles per row	0 = all that fit; else that many, spread or random	0
Output scale (mm/px)	As in 4a	Blank

7.3 The rows numbering

The rows are those of the row detection (7.5): with sown rows per plot set, the block of that many neighbouring rows with the highest contrast; otherwise the run of rows found. Row 1 is the one nearest the image top (rows running left-right) or the left edge (rows running up-down) of the plot image, before any turning. 'middle 3' takes the 3 central rows of that block, the same rows tab 4a tiles.

7.4 The recipe file

Every run writes PlotExtractor_recipe.json in its output folder, one section per step:

- plot_extraction: tab 3a or 3b - project, plots file, source images, margin, style, format, every option (written when the run starts).
- plot_tiles: tab 4a; rows_and_row_tiles: tab 4b - every field, the values passed to the run and the results (tiles, plots, tile size in pixels, output scale, row width).

A tiles run also copies the plot_extraction section of the run its crops came from (the recipe next to RawCrops), so the recipe in a tiles folder records the whole chain. **Load recipe…** on tabs 3a, 4a and 4b fills the tab's fields from a recipe file, to repeat a run or apply the same settings to another trial.

7.5 How the rows are found

The rows are found in every plot image by row_detect.py (the PhenoFuse 4.0 row method, adapted to raw photos, where the rows are not always parallel to the image edge; tab 1c runs the same method on the orthomosaic). Six steps: a greenness value per pixel, a threshold that separates plants from soil, a profile across the plot, the row spacing, the tilt, and a comb fitted to the rows. The figures show each step on OzBarley 2025 plot 1001 (a young crop, rows tilted 2.8° in the photo, 5 sown rows). The numbers are those the tool computed for this plot.

Step 1 · Greenness of every pixel (excess green)

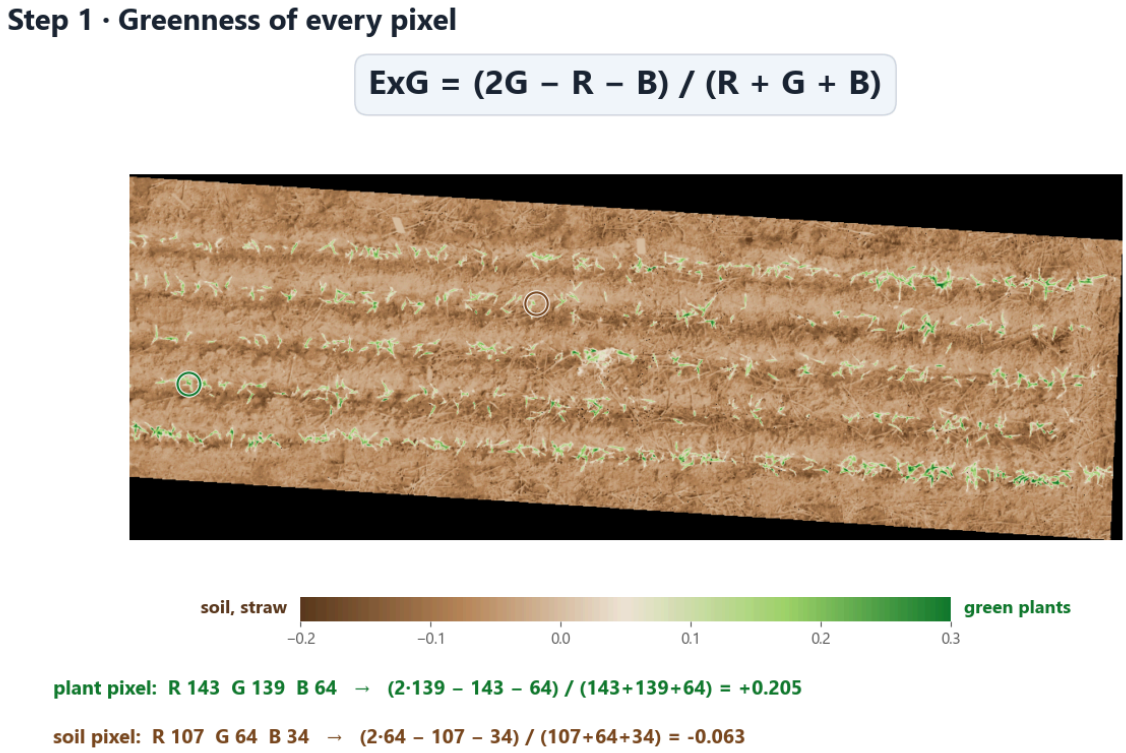

Figure 19. *The plot image turned into excess green: plants high (green), soil and straw low (brown). Two example pixels with their calculation.*

Every pixel gets **ExG = (2G − R − B) / (R + G + B)**. Dividing by R + G + B removes brightness, so a shaded leaf and a sunlit leaf score about the same. Green plants score high (about 0.1 to 0.3); soil and straw about 0 or below. Black pixels (outside a masked plot) are ignored.

Step 2 · Plant or soil: the Otsu threshold

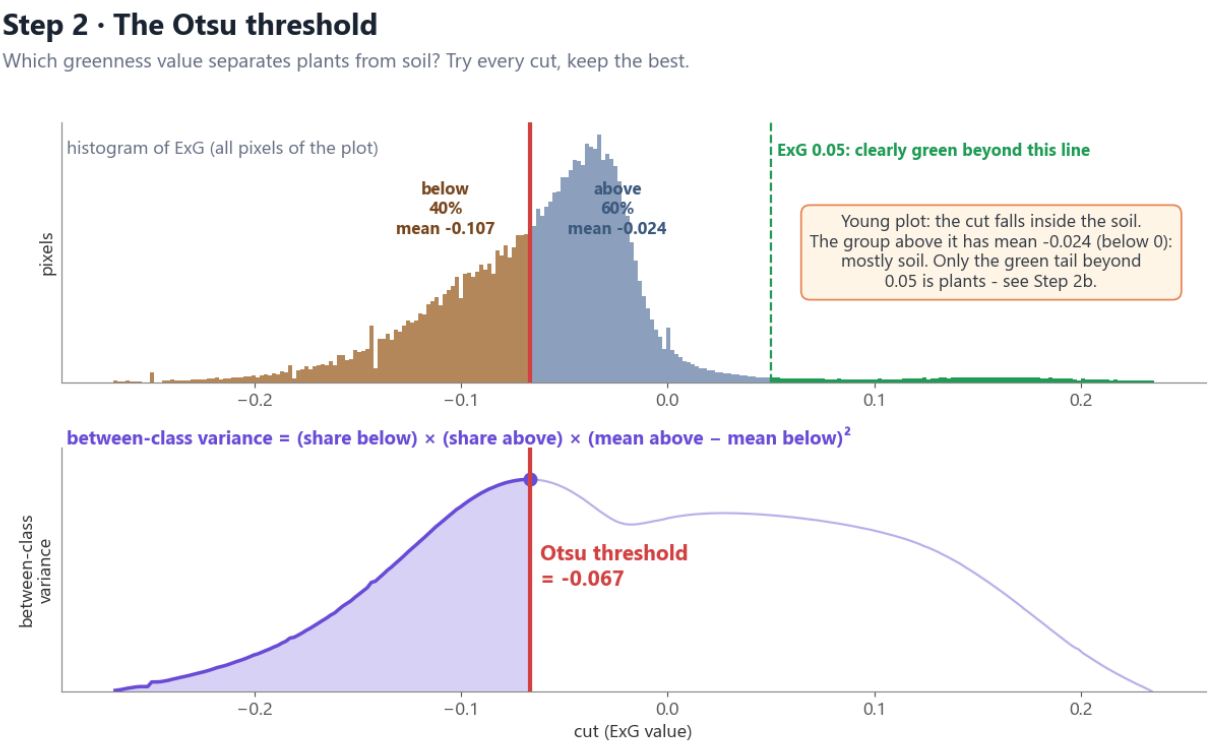

Figure 20. *Otsu's method on the ExG histogram of plot 1001: every cut is tried; the between-class variance (bottom) peaks at the threshold. Brown = below the cut; blue-grey = above the cut but not green (soil); green = clearly green, ExG above 0.05 (dashed line).*

Above which ExG value is a pixel a plant? **Otsu's method** answers this from the histogram alone, with no setting. It tries every possible cut. Each cut splits the pixels into two groups, below and above, and gets a score, the **between-class variance**:

between-class variance = (share below) × (share above) × (mean above − mean below)²

The score is large when both groups are big and their means are far apart. The cut with the largest score is the Otsu threshold: the cut that separates the histogram into the two most distinct groups. It adapts to every plot and every light condition.

What the colours show. Only the bars beyond the dashed line at ExG 0.05 are green: those pixels are clearly plants. On plot 1001 the plants are small, so there is no second hump for them, only a thin tail. Otsu still picks its best cut, at −0.067, inside the soil hump: the group above that cut (blue-grey plus the green tail) has a mean ExG of −0.024, below zero, so it is mostly lighter soil, not plants. Step 2b handles this.

Step 2b · When Otsu splits the soil (green fallback)

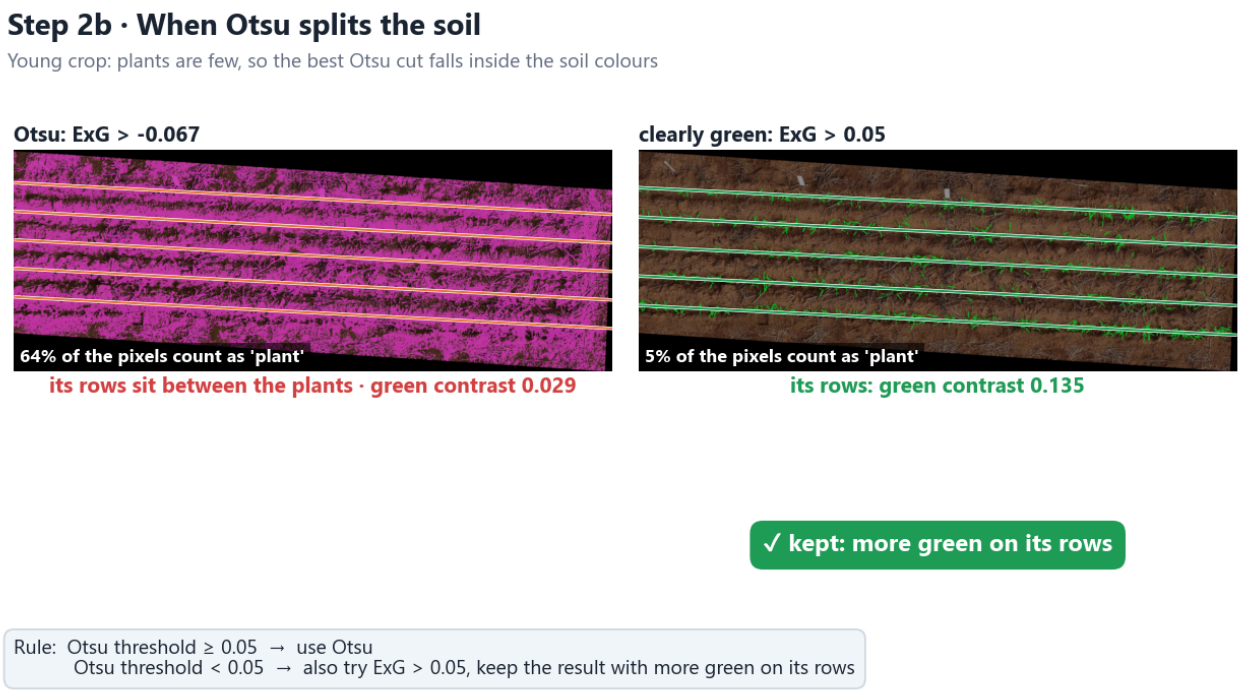

Figure 21. *Left: Otsu's cut (−0.067) falls inside the soil colours; 64% of the pixels count as plant and the rows found sit between the plants. Right: clearly green pixels (ExG > 0.05); its rows sit on the plants and are kept.*

Otsu always finds two groups, even when one of them is not plants. In a young crop the plants are a few percent of the pixels, so the best cut falls inside the soil colours: lighter soil and straw count as plants, and on ridged seedbeds the rows can then be found on the ridges, half a spacing off. So:

- **Otsu threshold ≥ 0.05:** the plants dominate the split; the Otsu result is used.
- **Otsu threshold < 0.05:** the rows are also found from the clearly green pixels only (ExG > 0.05). Both results are scored by how much more green sits on their rows than in the gaps (green contrast), and the higher one is kept. Plot 1001: Otsu 0.029, green 0.135, so the green result is used.

Step 3 · The row profile

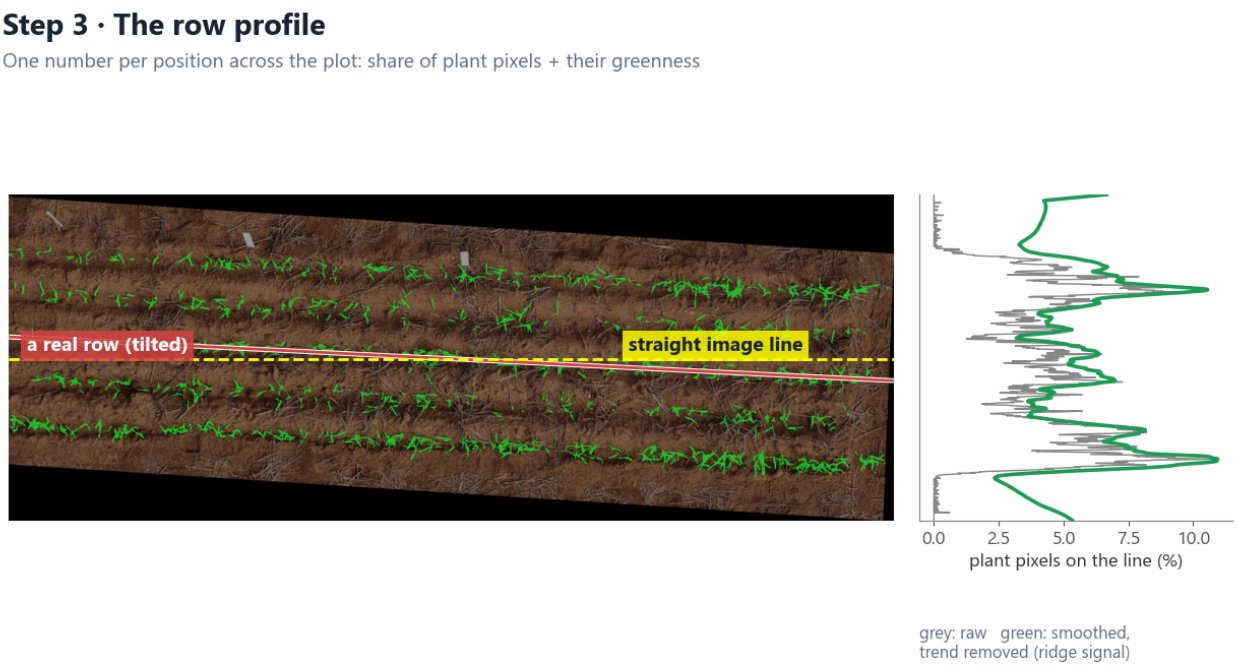

Figure 22. *Right: for each position across the plot, the share of plant pixels along that line (grey) and the smoothed ridge signal (green). The tilted rows smear the peaks of a straight image line.*

For each position across the plot one number is computed along a line in the row direction: the **share of plant pixels** on that line plus their **mean greenness** (the second helps at emergence, when seedlings are only a few pixels wide). Rows become peaks and the soil between them valleys. The profile is smoothed over 4 cm and the slow trend over 0.5 m is removed, so only the row pattern is left (the ridge signal). On a tilted plot a straight image line crosses several rows, so the spacing and the tilt are measured first.

Step 4 · Row spacing from short segments (Fourier)

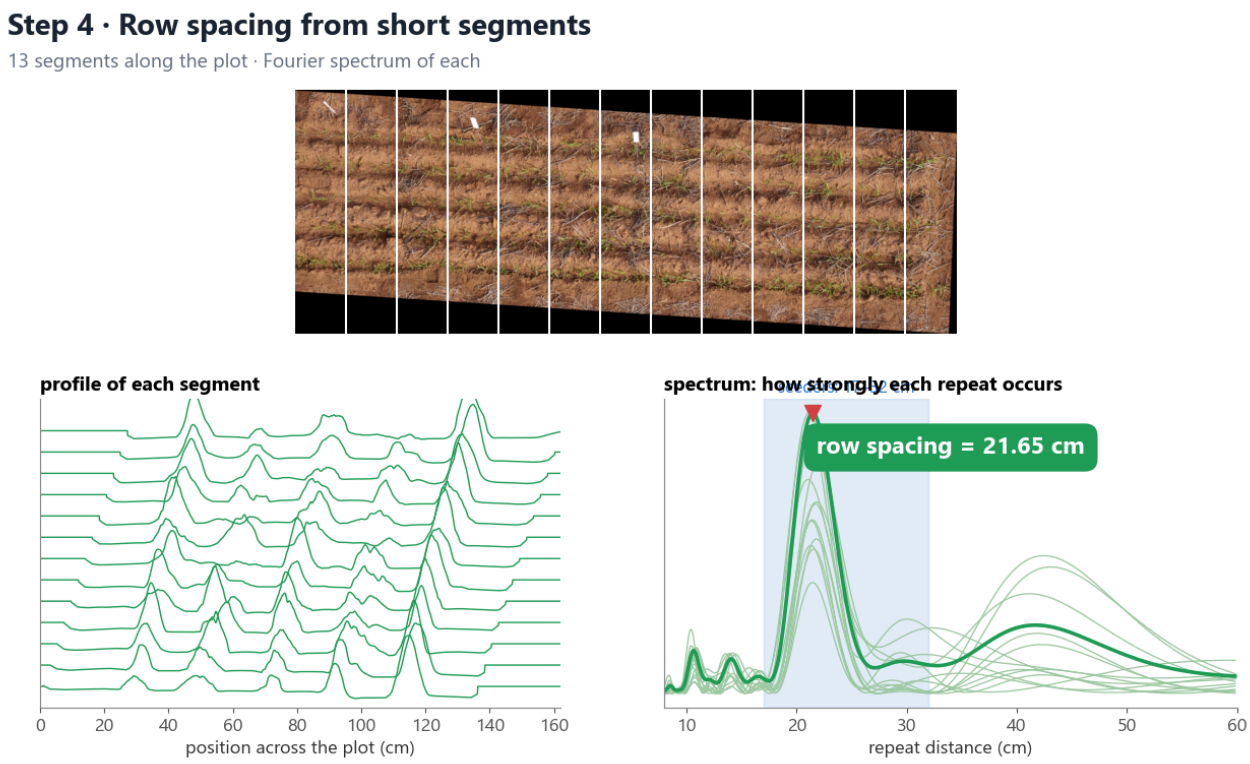

Figure 23. *The plot cut into 13 segments; the profile of each (left) and its spectrum (right). The sum of the spectra peaks at the row spacing.*

The plot is cut into 6 to 24 short segments along its length (13 here). Over a short segment even a tilted row stays sharp. Each segment's profile goes through a **Fourier transform**, which shows how strongly each repeat distance occurs (the position of the rows does not matter there, so the tilt cannot bias it). The spectra are added up and the strongest repeat between **17 and 32 cm** (the plot-seeder range; or exactly the row spacing if you enter one) is the row spacing. It is then refined on the whole, tilt-corrected plot: 21.65 cm here.

Step 5 · Tilt from the drift of the rows

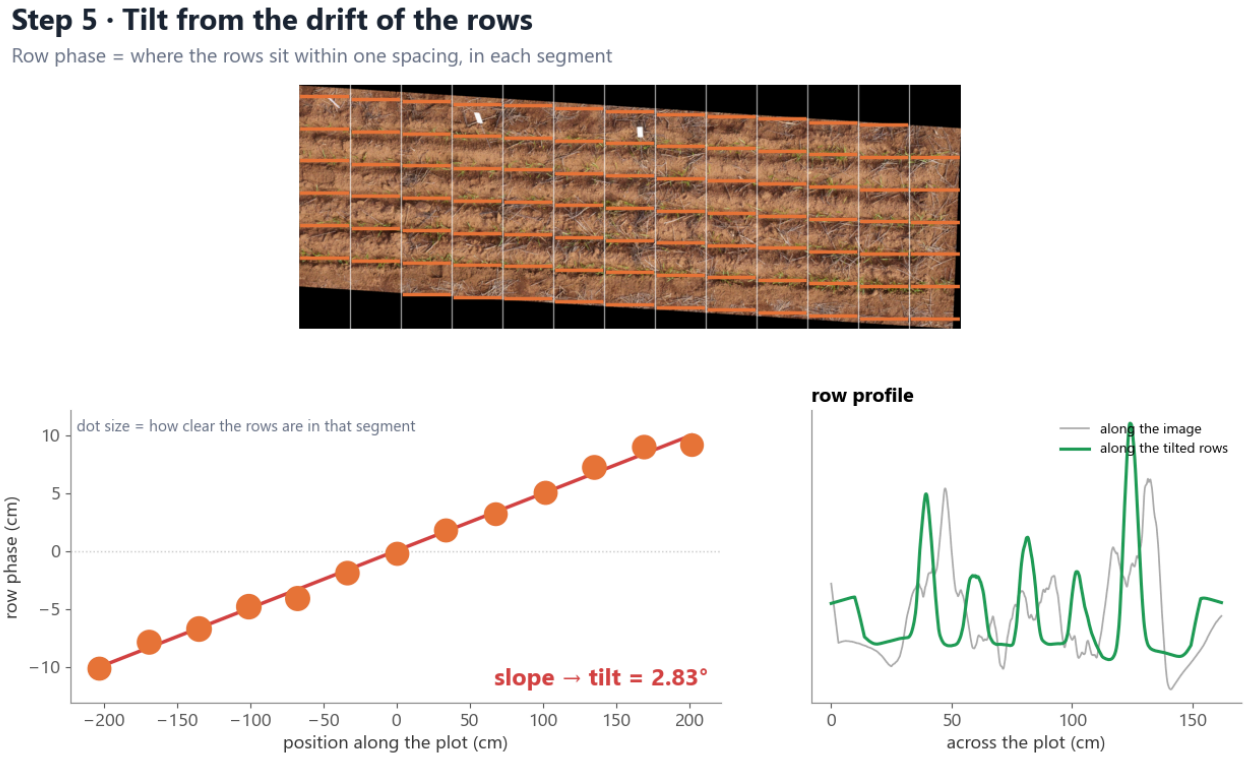

Figure 24. *Top: the row positions measured in each segment. Bottom left: their phase drifts steadily along the plot; the fitted line's slope is the tilt (2.83°). Bottom right: the profile before and after the tilt correction.*

In each segment the **phase** of the rows is measured: where the rows sit within one spacing (from the same Fourier component). If the rows ran exactly along the image, the phase would be the same in every segment; on a tilted plot it drifts steadily along the plot. A straight line is fitted through the phases (weighted by how clear the rows are in each segment, outliers dropped, 4 rounds); its slope is the **tilt** (a tilt above 8° is treated as no tilt). The profile is then measured along the tilted lines, and the row peaks become sharp.

Step 6 · The comb fit

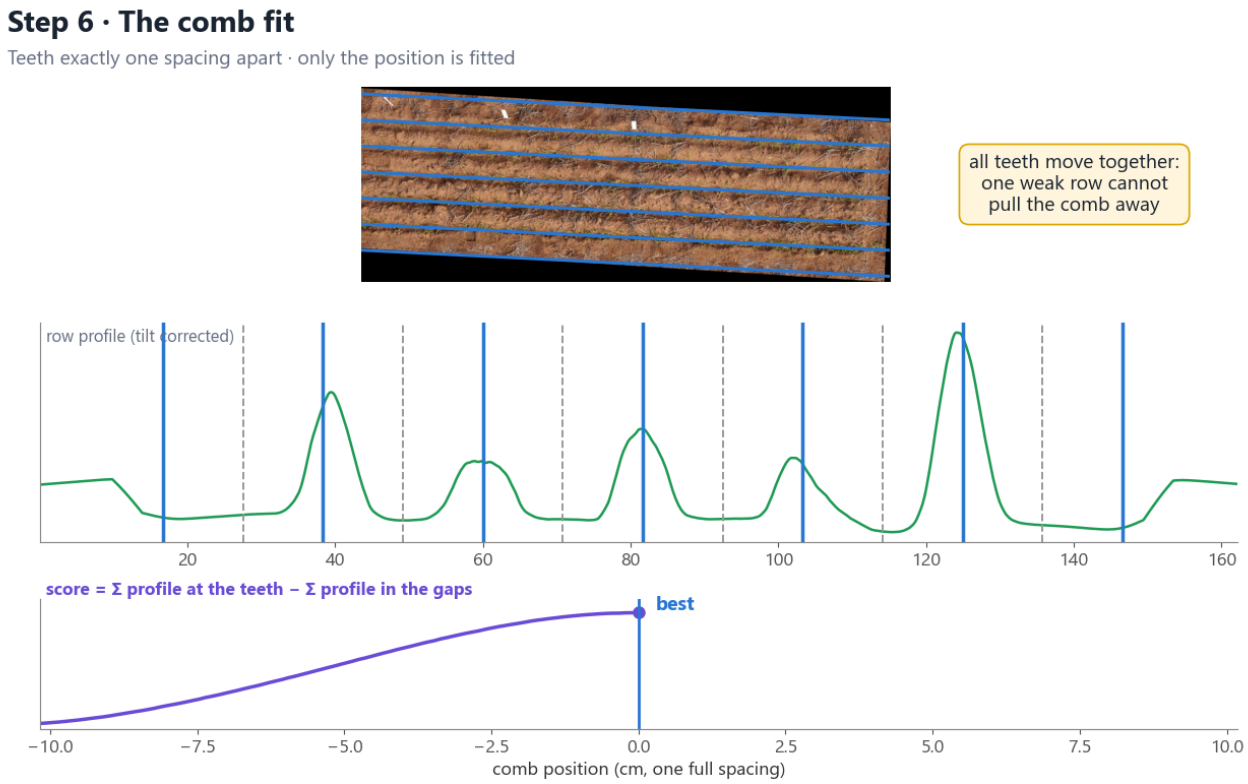

Figure 25. *The comb (blue teeth, one spacing apart) at its best position; dashed lines are the gaps between the teeth. Bottom: the score for every comb position across one spacing.*

A **comb** is a set of teeth exactly one row spacing apart, enough to fill the plot. The spacing is known, so only the comb's position is fitted: it slides across one full spacing, and every position gets a score:

score = Σ profile at the teeth − Σ profile halfway between the teeth

The score is high when the teeth sit on rows and the gaps on soil; the best position wins. Because all teeth move together, one weak, gappy or missing row cannot pull the comb away: the other rows hold it in place. That is why a comb is more reliable than picking the peaks one by one.

Which teeth are the plot's rows

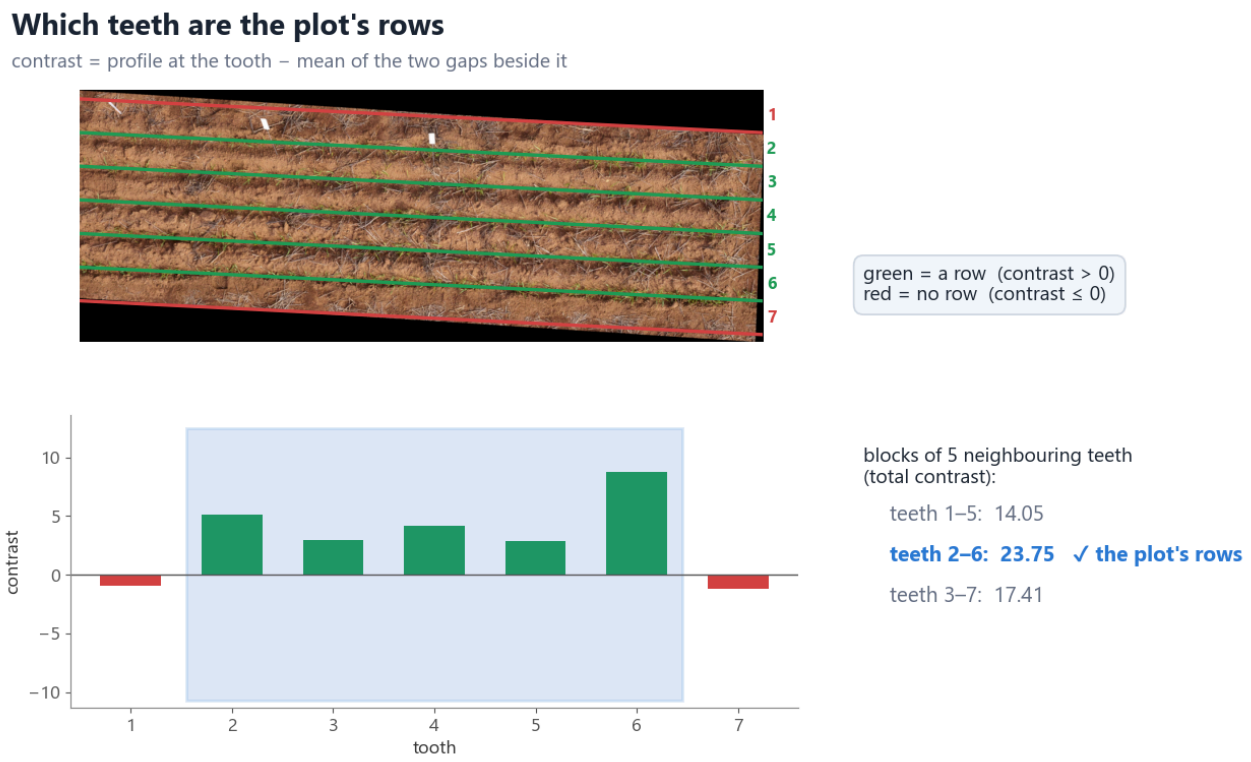

Figure 26. *Contrast of every tooth (green = a row, red = no row) and the block of 5 neighbouring teeth with the highest total contrast.*

Every tooth gets a **contrast**: the profile at the tooth minus the mean of the two gaps beside it. Positive = a real row (green), zero or negative = no row (red, e.g. bare ground at the edge of the crop). With **sown rows per plot** = 5, the block of 5 neighbouring teeth with the highest total contrast is the plot's rows (teeth 2-6 here); this keeps out rows of the neighbouring plots that the margin catches. With sown rows 0 the run of positive teeth is used, without the teeth cut by the crop edge.

Middle rows, band and tiles

Figure 27. *The middle 3 of the 5 rows (yellow), the band (half a spacing beyond the outer two, following the tilt) and the 0.5 m end zones (red). The tiles drawn are band-wide (versions before 2026-10-30); tiles are now 1 m × 1 m centred on the band (Figure in 7.7).*

The **band** of the N middle rows runs from half a row spacing outside the first middle row to half a spacing outside the last, and follows the tilt. The tiles (1 m, or sized to cover the rows) are centred on it, laid end to end inside the usable length (section 7.6). rows_report.csv lists the rows, spacing and tilt of every plot, and _rows_qa\ has a picture of each.

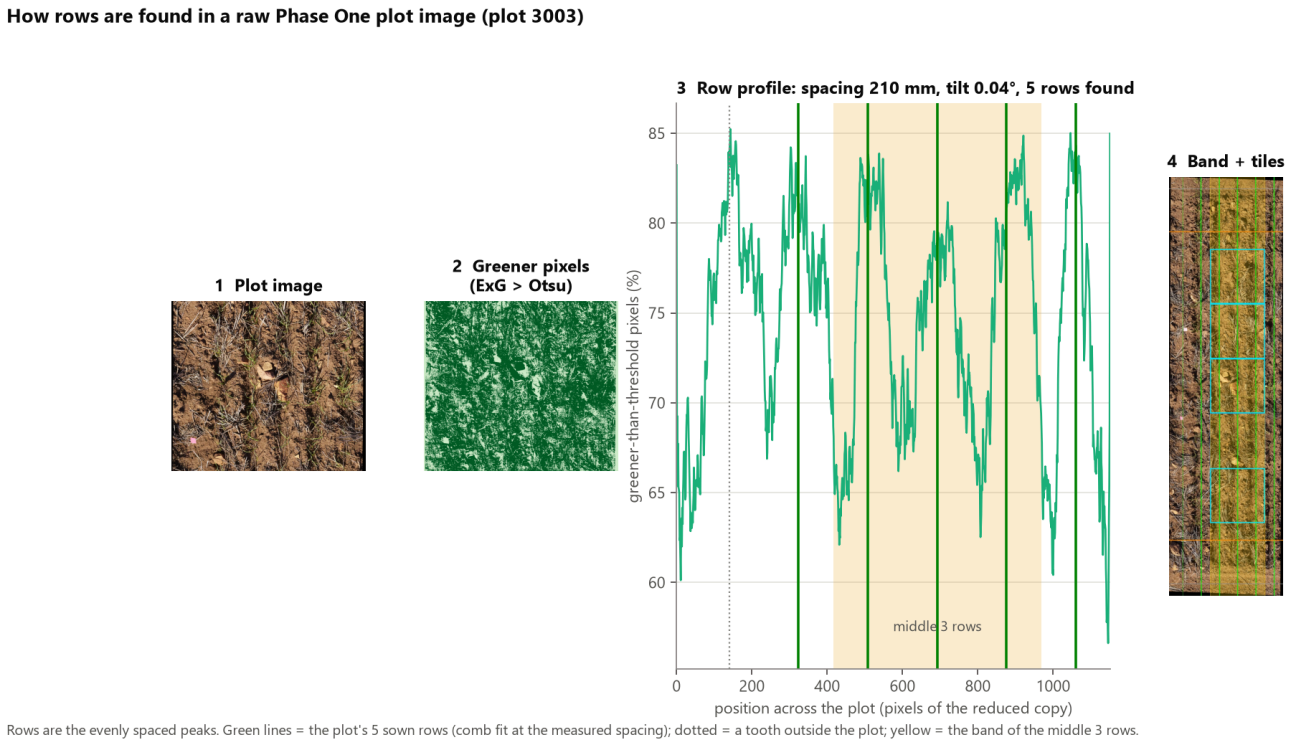

Figure 28. *Summary on another plot (York plot 3003): image, plant pixels, row profile with the fitted rows and the middle band, band and tiles.*

7.6 Where the tiles go along the plot

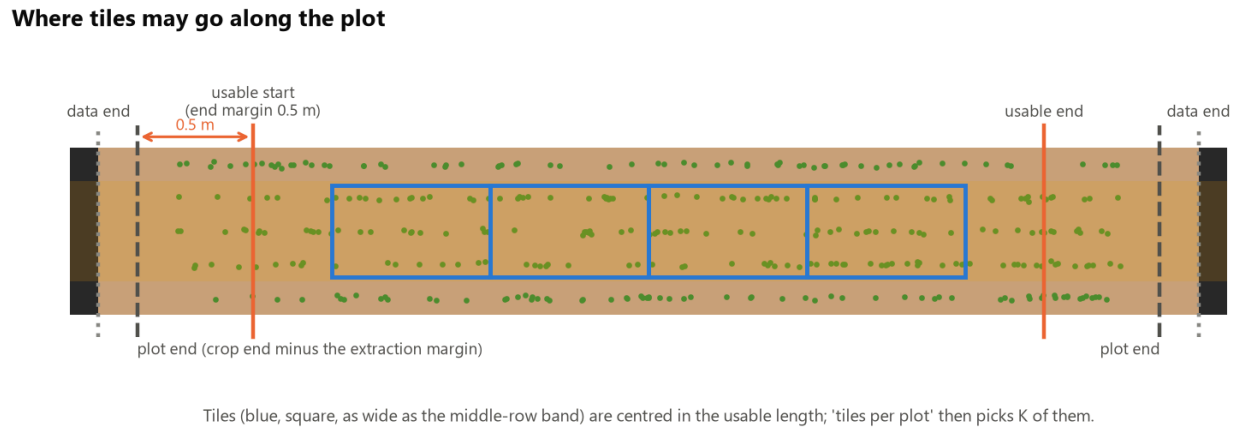

Figure 29. *Tiles stay inside the usable length and are centred in it.*

The plot ends are bare soil (and in a raw crop the black mask or the photo edge). Tiles there would teach a model that empty ground belongs to a plot. So the tiles stay inside the **usable length**: the plot ends (crop ends minus the extraction margin, read from manifest.csv), trimmed to the planted length where the green share clearly drops at the ends, minus the **end distance** you set. The tiles are centred in that length, then **Tiles per plot** picks how many are written.

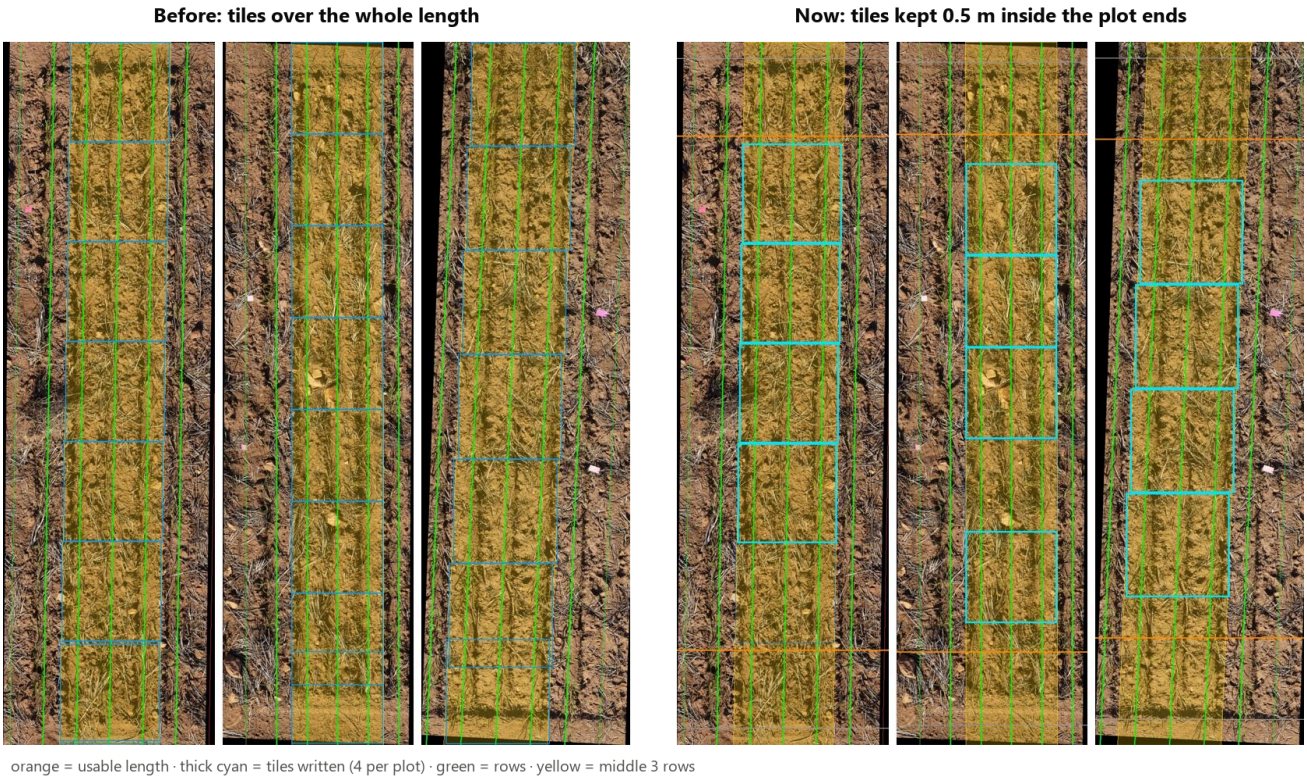

Figure 30. *Before the update tiles ran the whole plot length; now they stay 0.5 m inside the plot ends.*

7.7 What you get (tab 4a)

- <plot>\<plot>_tile_c<i>_r0.tif: the tiles, all the same size (tile size / output scale pixels); exact copies on plots at the output scale, resampled to that size on the others.

- tile_index.csv: per tile, the plot, the pixel window in the crop, the plot and tile mm/px, tile_m and resampled. Needed to map a detection back to its plot.
- rows_report.csv: per plot, rows found, spacing (px and mm), tilt, which rows were used, band, plot length, usable length, tiles that fit, tiles written.
- _rows_qa\<plot>_rows.jpg: green = rows, yellow = middle-row band, grey = plot ends, orange = usable limits, thick cyan = tiles written (numbered), thin blue = tiles not picked.

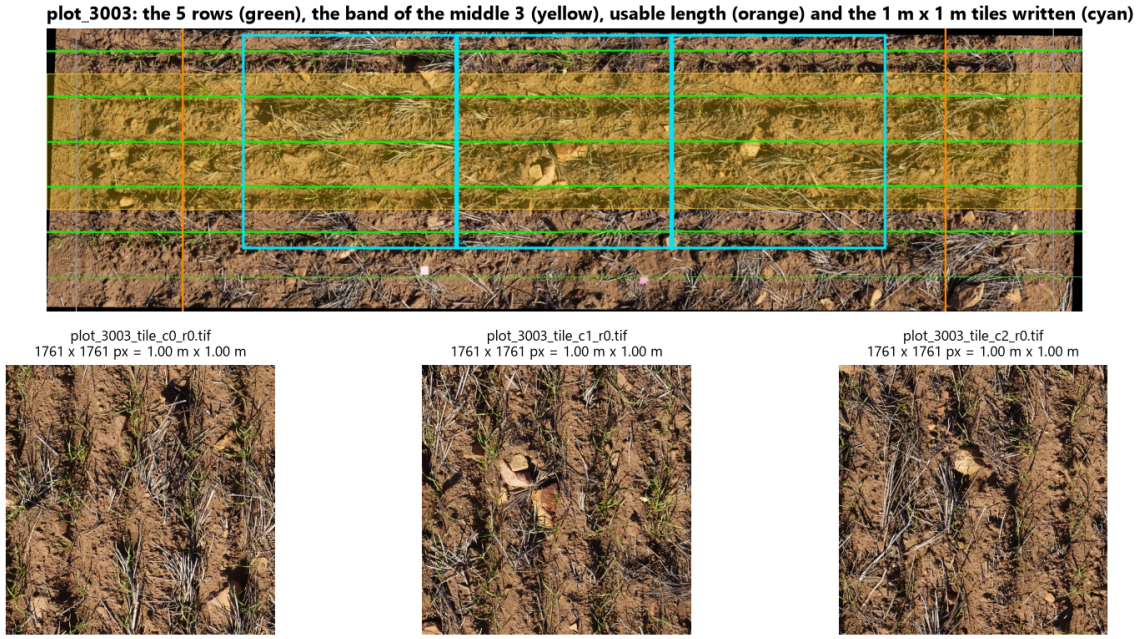

Figure 31. *Plot 3003 (raw): the middle 3 rows and the three 1 m × 1 m tiles that fit between the 0.5 m end zones, laid end to end and centred.*

7.8 Other tools on this tab

- **Plain grid tiles:** untick middle rows. Tiles of the same ground size with overlap over the seed-row band or the whole crop.
- **Rebuild one image per plot:** puts tiles (or labelled overlays with the same names) back together using tile_index.csv.

8. Tab 4c · QA, and are the images really original?

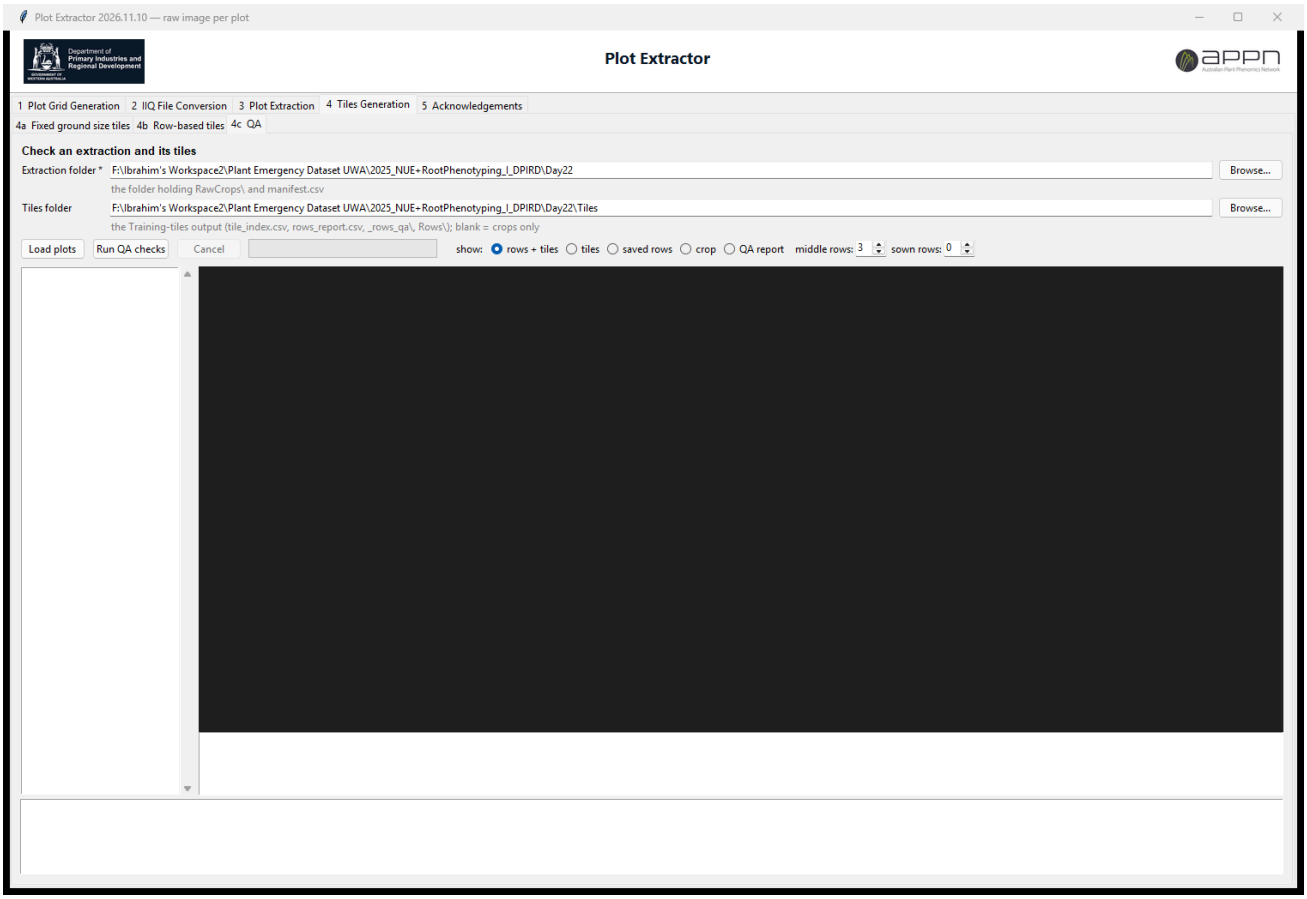

Figure 32. *Tab 4c, QA.*

Tab 4c checks an extraction (and its tiles): every raw crop against its source - the JPEG or TIFF box must be identical (a 16-bit TIFF source is compared at the crop's bit depth); an IIQ crop is compared with a fresh development of its box (IDENTICAL, or MATCH within 2/255 with noise reduction); a raw-sensor crop is read again from the IIQ and must be IDENTICAL - every tile against the crop, and the rows. It writes QA\QA_REPORT.txt with CSVs and shows per-plot pictures.

Tested on the York calibration trial (Phase One, 25 June 2025, 36 wheat plots):

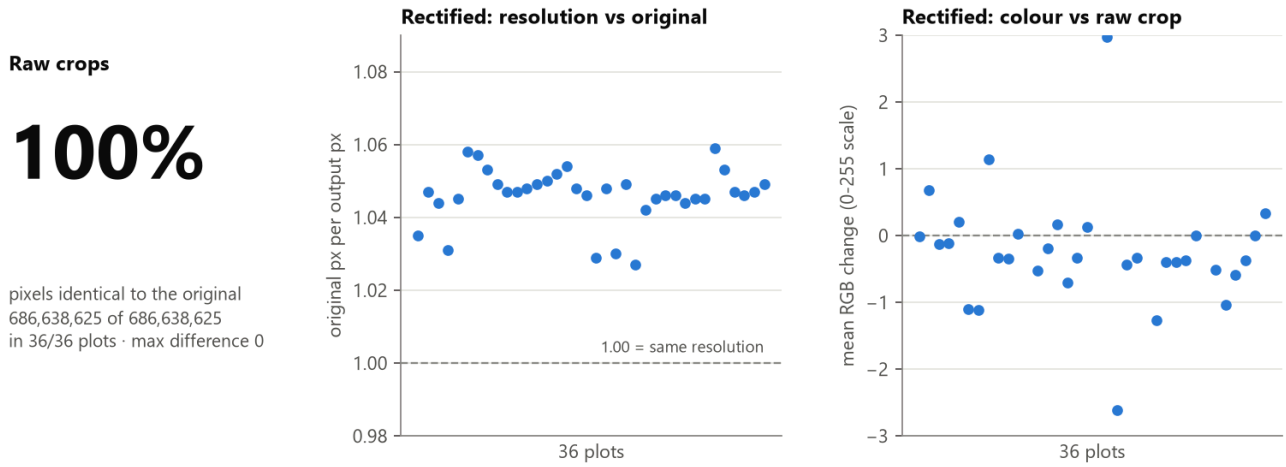

Figure 33. *Raw crops are identical to the original photos; rectified crops are resampled with unchanged colour.*

- **Raw (Masked to plot):** 36/36 plots, 686,638,625 pixels compared with the original JPEGs, every pixel identical. Colour profile copied byte for byte; lossless files.

- **Rectified:** resampled by design, about 1.05 original pixels per output pixel (~5% lower resolution); colour unchanged.
- **Tiles:** every tile is an exact copy of its window in the crop, so raw tiles are original pixels.

The check can be re-run on any extraction with fidelity_check.py (in the York output's fidelity\ folder).

9. Tab 5 · Acknowledgements

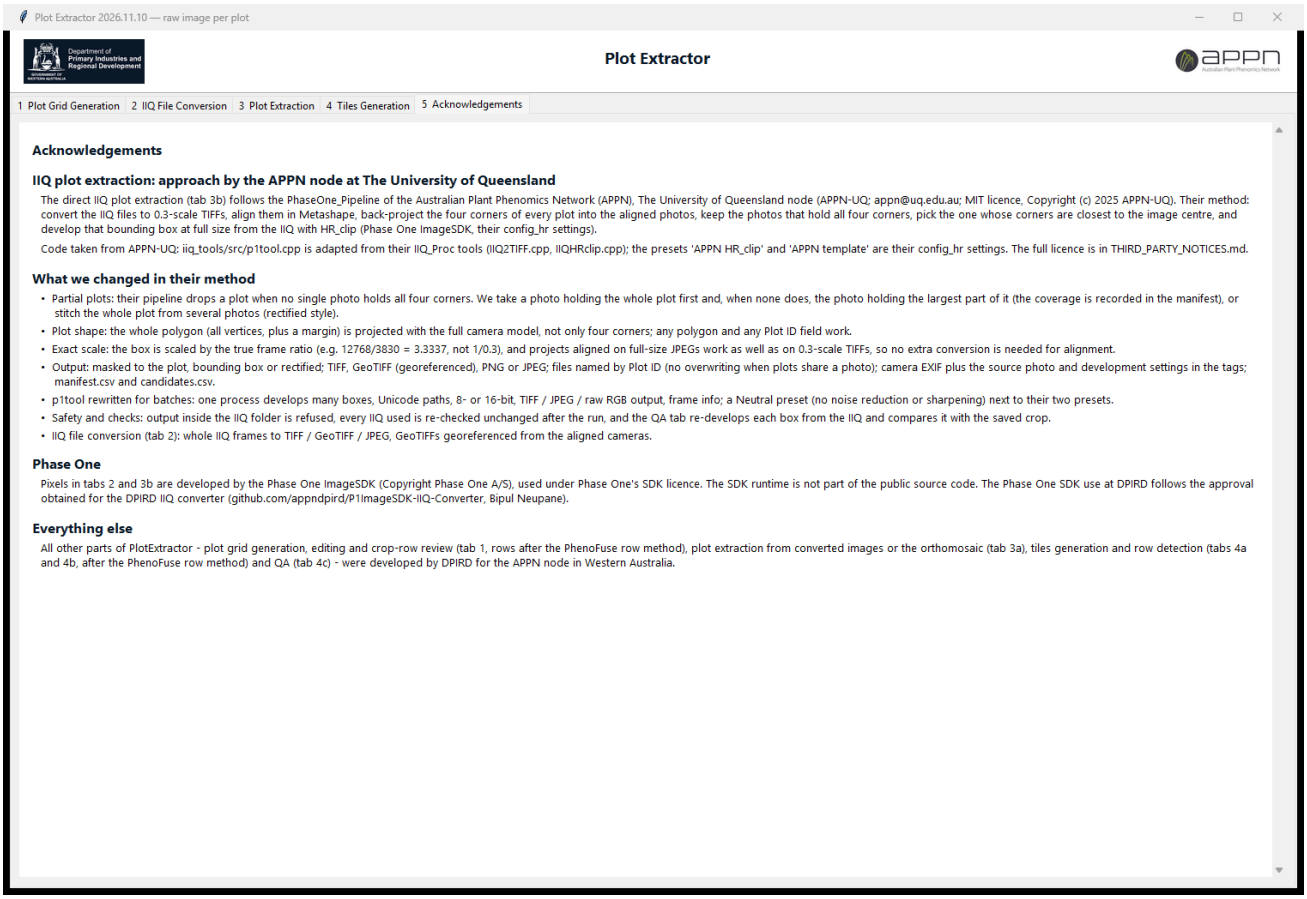

Figure 34. *Tab 5 lists the credits and our changes.*

The direct IIQ plot extraction (tab 3b) follows the **PhaseOne_Pipeline of APPN, The University of Queensland** (appn@uq.edu.au, MIT licence): back-projecting the plot corners into the aligned photos and developing the plot box from the IIQ with HR_clip. Pixels in tabs 2 and 3b are developed by the **Phase One ImageSDK** (Copyright Phase One A/S); raw sensor values are read with LibRaw (rawpy). Tab 5 lists what we changed in their method: partial plots (largest part) and stitching instead of dropping plots, the whole polygon with a margin, the exact frame scale, masked/rectified GeoTIFF output named by Plot ID, a batch p1tool with a Neutral preset, protection and re-checks of the IIQ files, QA, and whole-frame conversion with georeferencing. Everything else in PlotExtractor was developed by DPIRD for the APPN node in Western Australia.

10. Keeping the original data: formats, georeference and data loss

What changes the pixel values on the way from the camera to a plot image or a tile, and what does not. Measured on York 2025 (36 plots, direct IIQ extraction, bounding box, 0.10 m margin, every format; details in the technical report, experiment E8).

10.1 What each step does to the data

Step	Changes values?	York 2025 result
Raw sensor values (3b)	No - the recorded data	36/36 read back identical from the IIQ
Development (SDK, any preset)	Yes, by design: demosaic, colour, tone; APPN HR adds noise reduction and sharpening	APPN HR vs Neutral: mean difference 13.5/255; Neutral is repeatable bit for bit (6/6 plots), APPN HR within 2/255
16 → 8 bit	Yes: 65,536 levels become 256	8-bit file = 16-bit / 257 within 1 level; a plot uses ~65,000 green levels in 16 bit vs 256 in 8 bit
GeoTIFF vs TIFF vs PNG	No	36/36 plots pixel-identical, 8 and 16 bit
Deflate / LZW / no compression	No	Identical pixels; Deflate smallest (16-bit: 98 vs LZW 135 vs none 126 MB)
JPEG (quality 95)	Yes, lossy	PSNR 32.6 dB, mean error 4.3/255, up to 104/255, 99.9 % of pixels changed (noisy raw detail compresses badly)
Masked style	Only outside the plot (set to 0)	Pixels inside the plot unchanged
Rectified style	Yes, resampled	~1.05 original pixels per output pixel
Tiles, unit px	No	664/664 tiles exact copies
Tiles, unit m / row tiles	Yes on most plots (resampled to one scale)	1 m tiles: 10 exact, 362 resampled (× 0.89-1.05)

10.2 How the GeoTIFF is georeferenced, and is anything lost?

A raw plot crop is a box of a perspective photo, not an orthophoto. PlotExtractor does **not** warp it to make it a map: the pixels are written exactly as cut (or developed). Only the **georeference** (metadata) is added:

1. A grid of points over the crop (about 5 x 5 per plot; 23 on York) is projected from the photo onto the DEM with Metashape's camera model (lens distortion included): each crop pixel (column, row) gets its ground X, Y, Z - **ground control points** (GCPs).
2. The GCPs are transformed into the output CRS (EPSG:7850 by default).
3. An **affine transform** (6 numbers: origin, pixel size, rotation, shear) is fitted through them by least squares and written as the GeoTIFF geotransform, so GIS software places the image directly. The GCPs themselves are kept in the PE_GCPS tag and the fit residual in PE_AFFINE_RESID.

Data loss: none in the pixels. The GeoTIFF, plain TIFF and PNG of the same crop are pixel-identical (36/36 York plots). The affine is a straight-line approximation of a perspective view over uneven ground, so a **position** read from the georeference is approximate: on York the fit residual was 4.7 mm on average (median of plots) and at most 20 mm, at 0.50-0.60 mm per pixel. That is the accuracy of placing the image on a map, not a change of the image. For exact ground positions use the GCPs in PE_GCPS (or the orthomosaic); for exact pixels use the file as it is. A GeoTIFF is **not** resampled; only the Rectified style and metre tiles resample.

10.3 Recommended settings for no data loss

Goal	Settings
The recorded data, nothing changed	Tab 3b, Pixels = Raw sensor values, Bounding box (or Masked), GeoTIFF, Deflate
Developed images with the least processing and the full tonal range	Tab 3b (or 3a from 16-bit TIFFs of tab 2), Neutral, 16 bit, Bounding box or Masked, GeoTIFF, Deflate
Best-looking developed images	Tab 3b, APPN HR_clip, 8 or 16 bit, Masked, GeoTIFF, Deflate
Tiles with the original values	Tab 4a, unit px (e.g. 1024), Bits same as plot image, tif, Deflate
Avoid	JPEG output, Rectified style and metre tiles when the values themselves matter

10.4 The APPN PhaseOne_Pipeline on the same data

The original APPN-UQ pipeline (03_backprojection.py + HR_clip.exe, unmodified) was run on the same York flight. It made clips for **30 of 36 plots**: it drops a plot when no single photo holds all four corners (1003-1006, 6001, 6002), where PlotExtractor takes the whole plot from another photo (2 plots) or the photo with the largest part (96-99.6 %, 4 plots). Its clips are 8-bit, uncompressed TIFFs without georeference, the axis-aligned box of the four corners (85-99 % plot). Where both tools chose the same photo (22 plots), their box lay inside PlotExtractor's, and developing **their box with their settings file through PlotExtractor** gave identical pixels on 19 of 22 plots (the other 3 differ on a few pixels from the SDK's non-repeatable colour noise reduction). The settings file now in their repository (config_hr.txt = PlotExtractor's "APPN template" preset) raises exposure by 0.8 EV with highlight / shadow recovery: images ~40 levels brighter than Neutral and 1.7 % clipped pixels (vs 0.2 %), i.e. data lost in the highlights. Running it took 54 s (back-projection) + 13 s (HR_clip).

11. Tips and good practice

11.1 Test first

Run Limit = 3, open the crops and the tile QA pictures, then run everything.

11.2 Use a small margin

A plot grid is rarely exactly on the rows (at York it was about 9 cm off). A 0.10 m margin keeps the outer rows whole, which makes the row choice reliable, and does not add neighbouring rows when the inter-plot gap is ~40 cm.

11.3 Give the sown rows

With the number of sown rows set, all 36 York plots used rows 2-4 of 5 in both the raw and rectified sets. Without it, a row cut by the plot edge can shift which rows count as the middle ones.

11.4 Flight planning for whole plots in one photo

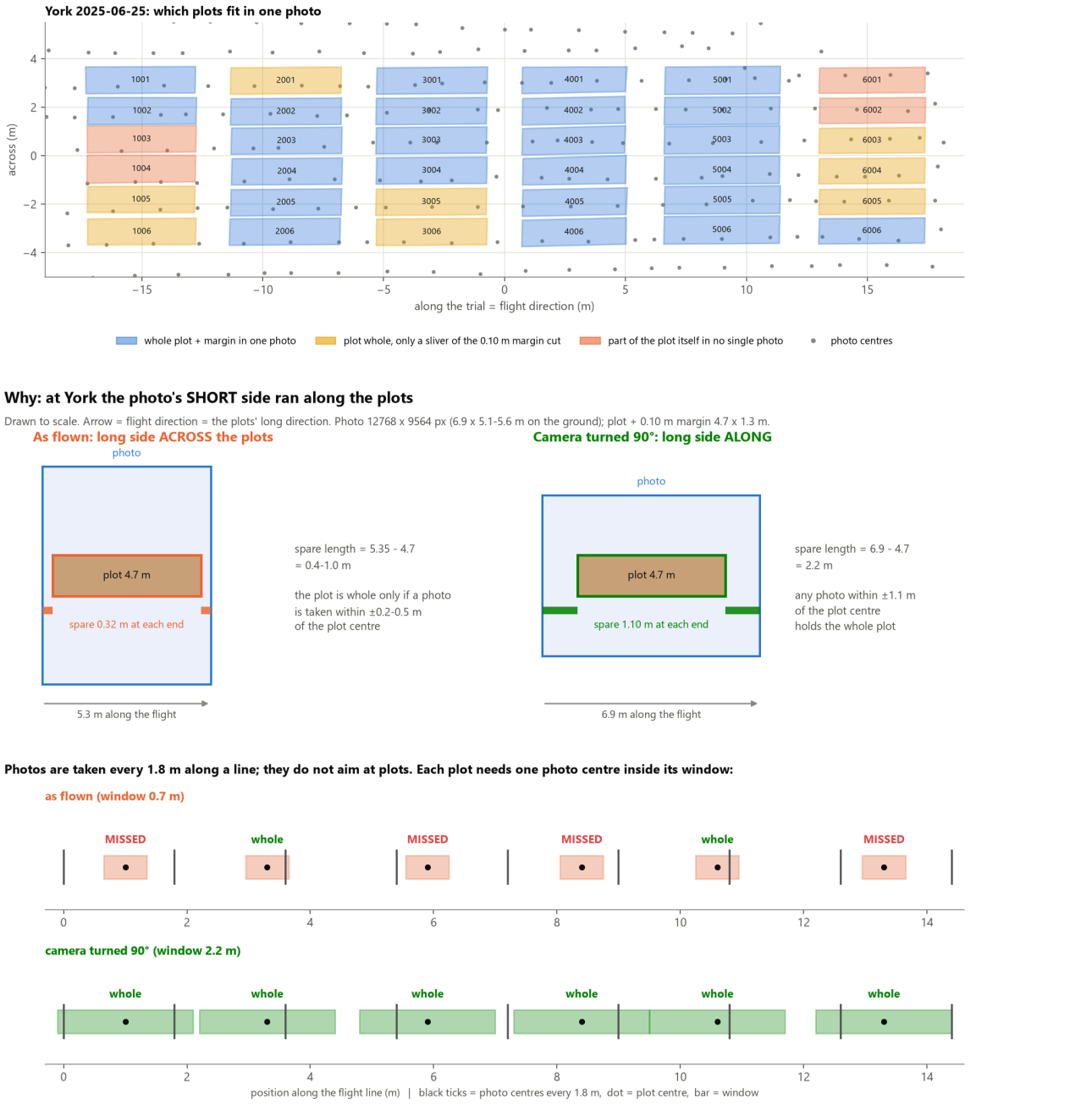

Figure 35. *Top: York plots coloured by fit. Middle: one photo and one plot drawn to scale, as flown and with the camera turned 90°. Bottom: each plot needs a photo centre inside its window; photos come every 1.8 m.*

At York (25 June 2025) the flight lines ran along the plots, but the photo's **short side** (9564 px, 5.1-5.6 m on the ground) lay along them and the long side (12768 px, 6.9 m) across. A plot plus its 0.10 m margin is 4.7 m long, so a photo was only 0.4-1.0 m longer than the plot: the **spare length**. The whole plot is in a photo only if that photo was taken within half the spare length (±0.2-0.5 m) of the plot centre: the plot's **window**.

The camera does not aim at plots: it takes a photo every **1.8 m** along the line (median of 762 photo pairs). A window narrower than 1.8 m catches a photo centre only by luck. Most plots were lucky on at least one flight line; 4 of 36 never were, so their raw images are cut at the photo edge.

Rule: spare length > distance between photos. Spare length = photo length along the plots − plot length (with margin). Then at least one photo centre always falls inside every plot's window, on every flight line.

- **Turn the camera 90°** (rotate it about the vertical axis) so the photo's long side runs along the plots: 6.9 − 4.7 = 2.2 m to spare, more than the 1.8 m between photos, so every plot is whole in at least one photo on every line.
- **Check a flight before processing:** open one photo. The plots must run along its **long** side. (Pointing the camera straight down, −90° gimbal pitch, is a different setting and does not change this.)
- **Or take photos more often:** the distance between photos must be below the spare length (here below 0.4-1.0 m, usually not practical).
- **Or fly 2-3 m higher** (slightly lower resolution).
- **Start and end flight lines a few metres past the trial** so the end plots get the full number of photos.

11.5 Check the QA pictures

Open a handful of _rows_qa pictures: the green lines should sit on the rows, the yellow band on the middle rows and every cyan tile between the orange lines (4a), or every cyan row tile centred on a green line (4b).

11.6 Use one output scale across trials

Within one run every tile has the same pixel size. Runs on different flights have different median scales, so for a training set built from several trials set the same Output scale (mm/px) in every run (e.g. 0.57); the recipe records the value used.

12. Troubleshooting

Message or symptom	Cause and fix
No valid Metashape license	Licence server not reachable: connect to the network/VPN.
plots are ~X km away from the project centre	Wrong CRS or wrong .psx. Check the shapefile's .prj or set Plot CRS.
raw file missing	Set Raw images folder to where the photos are (the JPEG folder).
Chunk 'X' not found	The chunk saved from another project: pick the chunk again (or (active)) after choosing the .psx.
Phase One SDK status not OK (tabs 2, 3b)	Leave the SDK folder blank to use the bundled SDK, or point it at an SDK with ImageSdkCpp.dll and SensorProfiles.
No .IIQ file matches the cameras (tab 3b)	The IIQ folder must hold the flight's raw files with the same names as the photos in the project (T0_Raw).
frame left without georeference (tab 2)	The camera is not aligned in the chosen chunk, or the project/chunk is not the one the photos were aligned in.
raw_covered_pct below 100	No single photo holds the whole plot. Use the rectified image for that plot, or see 11.4.
no middle-row tiles (only K rows found)	Rows not visible enough or too few rows. Check the QA picture; set sown rows; use a larger extraction margin.
usable length shorter than one tile	Plot too short for the tile size plus end distance: reduce 'keep tiles … m from the plot ends' or the tile size.
smaller than one 1 m tile - skipped / plot is N px across, one tile is M px	The plot image is smaller than one tile in that direction: use a smaller tile size.
rows [k] not in this plot (4b)	A listed row number is beyond the rows found; check sown rows per plot and the QA picture.
no ground scale (4a/4b)	The plot images are not georeferenced (plain TIFF / PNG / JPEG): extract as GeoTIFF, give the plot width (mm), or use tile unit px.
has 1 band(s): raw sensor values (tab 4)	Raw sensor crops are not RGB images: tile developed crops (3a, or 3b with Pixels = Developed RGB).
16-bit output needs 16-bit sources (3a)	The raw images are 8-bit (JPEG or 8-bit TIFF): convert the IIQs to 16-bit TIFF in tab 2, or use tab 3b.
the orthomosaic is striped (1c)	Reading plots is slow: make a tiled copy (Make tiled copy, tab 3a) and use it as the orthomosaic.
only K tiles fit, N asked	The plot holds fewer tiles than asked; all that fit are written.
Middle rows look off by one row	Set the sown rows per plot and re-extract with a 0.10 m margin.

Appendix: files

File	Contents
manifest.csv	plot_id, plot_area_m2, margin_m, raw_image, raw_file, crop_w/h, corners_in_frame, raw_covered_pct/m2, cand_frames, best_frame, best_cover_pct/m2, frames_used, seam_y_pct, ortho_*, source_path, raw_style, src_x0, src_y0, src_x1, src_y1, note
candidates.csv	plot_id, rank, camera, cover_pct, cover_m2, corners_in_frame, aniso, mm_per_px, used
tile_index.csv	tile, rel_path, plot_stem, col, row, x0, y0, w, h, out_px, src_mm_px, tile_mm_px, ground_mm, tile_m, resampled, band_lo, band_hi, across_axis, src_w, src_h, veg_pct, src
row_tiles_index.csv (4b)	tile, rel_path, plot_stem, row, of_rows, tile_k, x0, y0, w, h, out_w, out_h, turned_90, resampled, src_mm_px, out_mm_px, tile_len_m, row_width_m, row_centre_px, angle_deg, row_lo_px, row_hi_px, veg_pct, src, bits, src_bits, georef, source photo link
<grid>_rows.shp / .csv (1a, 1c)	One line per crop row: Plot_ID, row_idx, v_off_m (offset from the plot centre across the rows), length_m, angle_deg, source (count / detect / edit), contrast; the CSV lists rows per plot, spacing and positions
PlotExtractor_recipe.json	The settings of each step (plot_extraction, plot_tiles, rows_and_row_tiles) with the results
rows_report.csv	plot_stem, src, ok, reason, rows_found, rows_present, pitch_px, pitch_mm, angle_deg, middle_rows, band_lo, band_hi, plot_len_m, end_trim, usable_lo, usable_hi, usable_len_m, tiles_fit, tiles_in_band, tiles_drawn